📖 BOOKHIVE

An Indian Online Bookstore

Complete Project-Based Teaching Guide

Build a Real Website from Scratch

Using HTML, CSS & JavaScript

Trainer's Reference & Student Handbook

Topic-by-Topic | Step-by-Step | Industry-Aligned

About This Report

This is a complete project-based teaching guide for instructors. Instead of teaching HTML, CSS, and JavaScript as separate isolated topics, you'll teach them in the FLOW of building a real website. Students learn each concept WHEN they need it, not before.

The project is 'BookHive' - an Indian online bookstore. It's relatable (everyone has bought books online), covers every essential web development concept, and produces a portfolio-worthy result. By the end, every student walks away with a real, deployed website they can show in interviews.

How to Use This Guide

- **Per Topic Teaching:** Each phase introduces a topic, shows the project context, then walks through the code.
- **Teaching Moments:** Marked with □ - these are key concepts to slow down and emphasize.
- **Code Listings:** Type-and-go code blocks students can follow live.
- **Progressive Build:** Each phase adds to the previous - by Phase 12, the site is complete.

Why Project-Based Teaching Works

Students don't remember 'all the HTML tags' from a slide. They remember 'we used <article> for each book card on BookHive'. Project-based teaching creates memory anchors. Each concept lives in a context they built with their own hands. This is how senior engineers learn - by doing.

> **Course Duration with This Guide**
>
> This project naturally spans 5-6 weeks of training (about 25-30 sessions of 2 hours each). It maps perfectly to the first 3 weeks of the MERN Stack Course curriculum, just structured project-first rather than topic-first.

Table of Contents - 12 Phases

PHASE 1: PLANNING & SYSTEM ARCHITECTURE

Why Plan Before Coding?

Before any developer touches code at Razorpay or Flipkart, they spend hours PLANNING. They draw the system, sketch the UI, list features, decide the structure. Why? Because writing code without planning is like building a house without a blueprint - you'll demolish and rebuild constantly.

Today, we plan BookHive. Tomorrow, we code with clarity.

Project Vision - What is BookHive?

BookHive is an online bookstore where Indian readers can browse, search, and order books. Think of it as a smaller, focused Amazon for books with personality - clean design, easy navigation, and a focus on Indian authors and bestsellers.

Target Users

- College students looking for textbooks and novels.
- Professionals buying business and self-help books.
- Casual readers exploring fiction, biographies, and Indian literature.

Core Features (MVP - Minimum Viable Product)

- Home page with featured books and categories.
- Books listing page with search and filter.
- Individual book detail page.
- Shopping cart with quantity control.
- Login and signup forms.
- About and Contact pages.
- Responsive design (works on mobile, tablet, laptop).

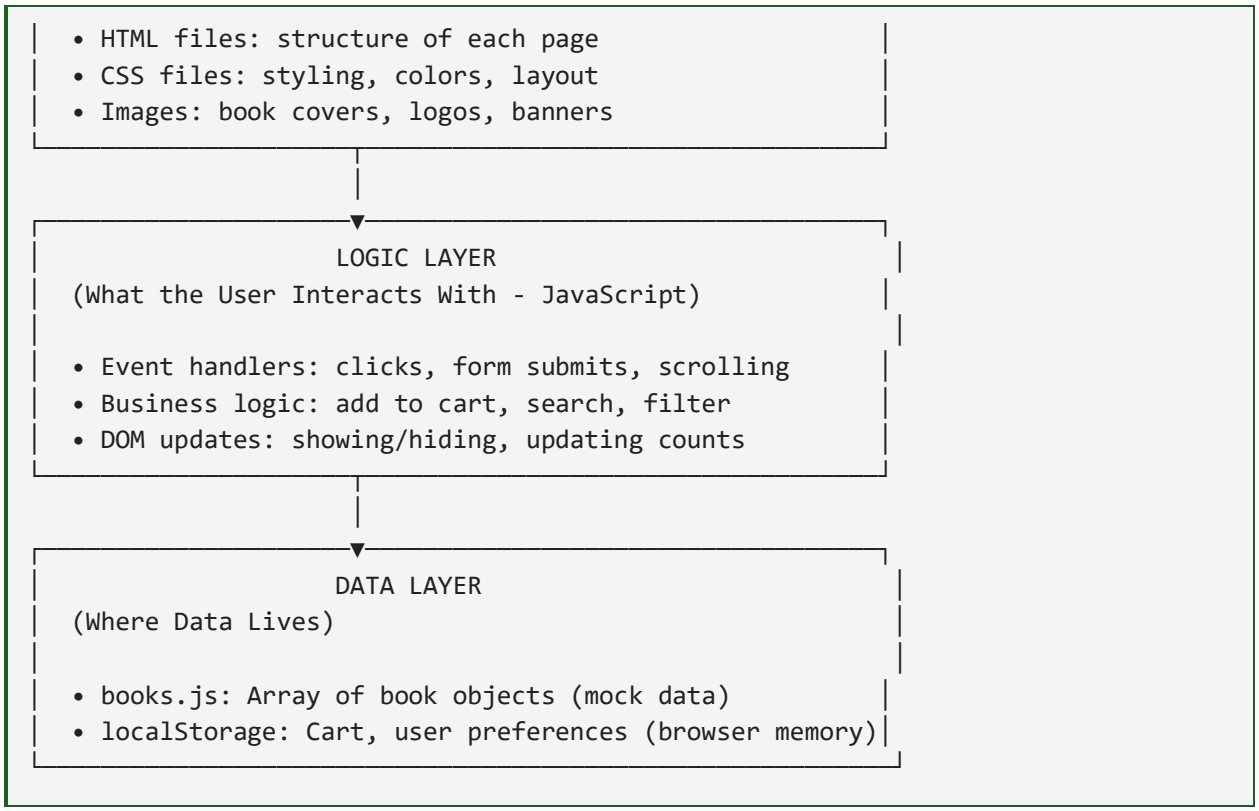
? Teaching Moment: Always start with MVP - Minimum Viable Product. Build the SIMPLEST version that works. Then add features. Many beginner developers try to build everything at once and finish nothing.

System Architecture (For This Project)

Since this is a frontend-only project (no real database or server), our architecture is simpler than full-stack apps. But the principles are the same.

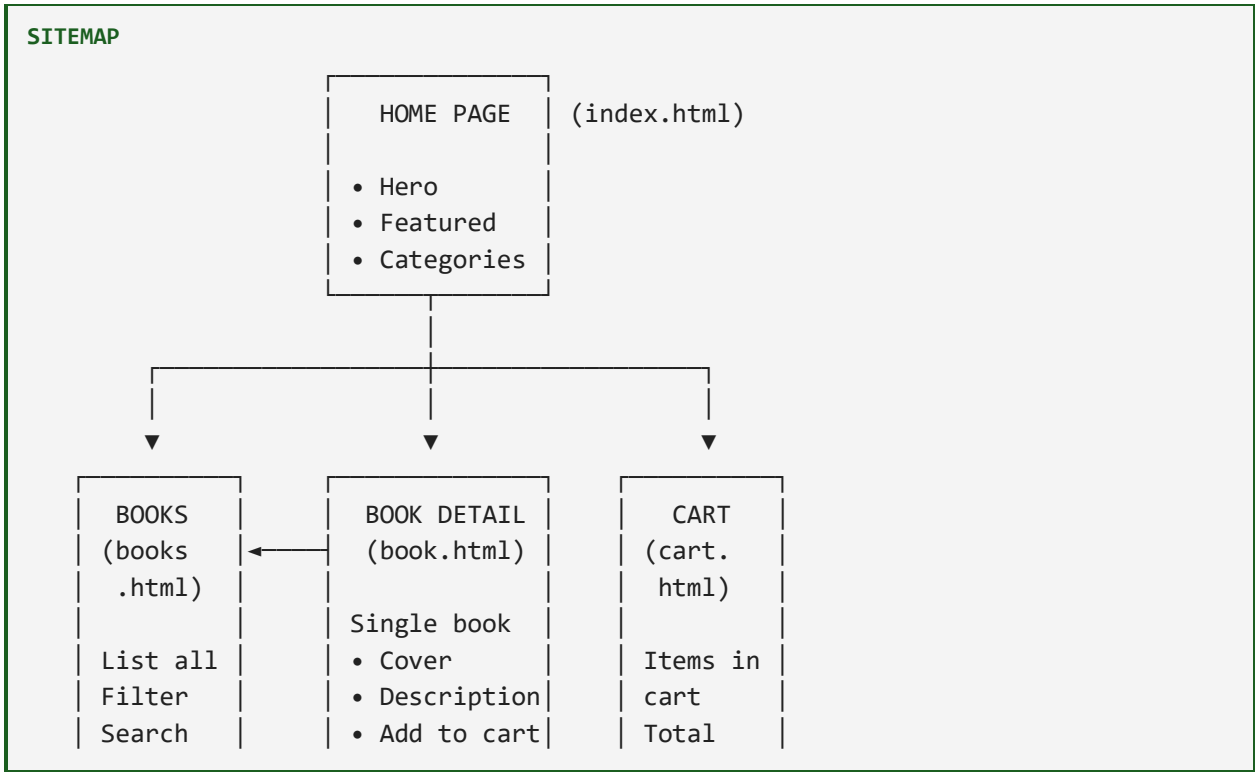
The Three Layers of Our Website

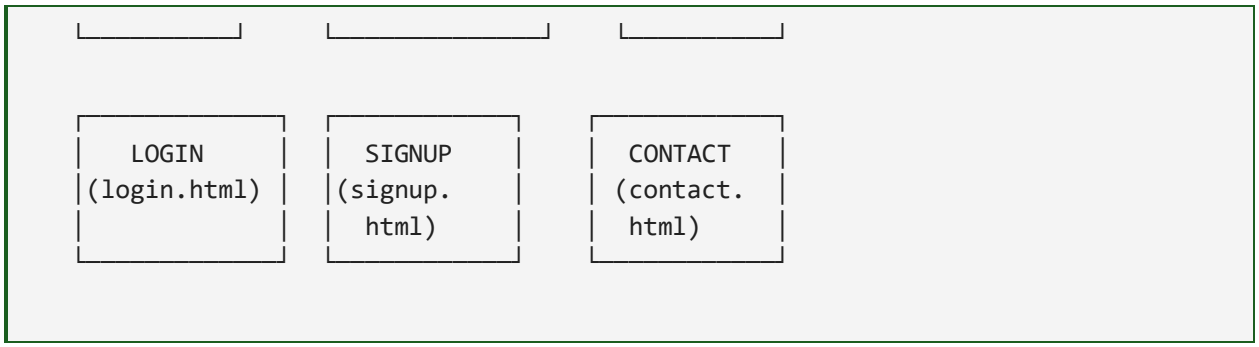
```
ARCHITECTURE
 ┌──────────────────────────────────────────────────────────┐
 │                    PRESENTATION LAYER                    │
 │  (What the User Sees - HTML + CSS)                       │
 │                                                          │
```



Teaching Moment: This is called 'Separation of Concerns'. HTML handles structure, CSS handles look, JS handles behavior. Mixing them creates chaos. Senior developers obsess over this separation.

Page Architecture - The Site Map

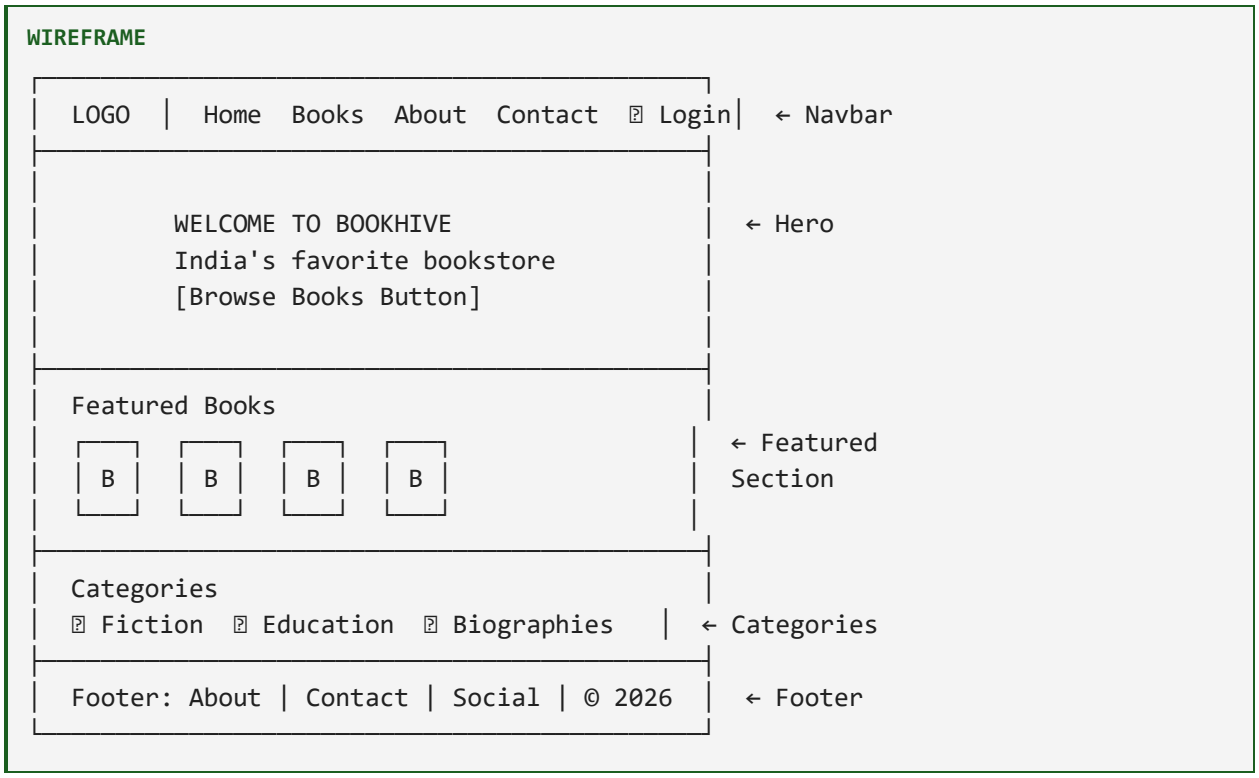




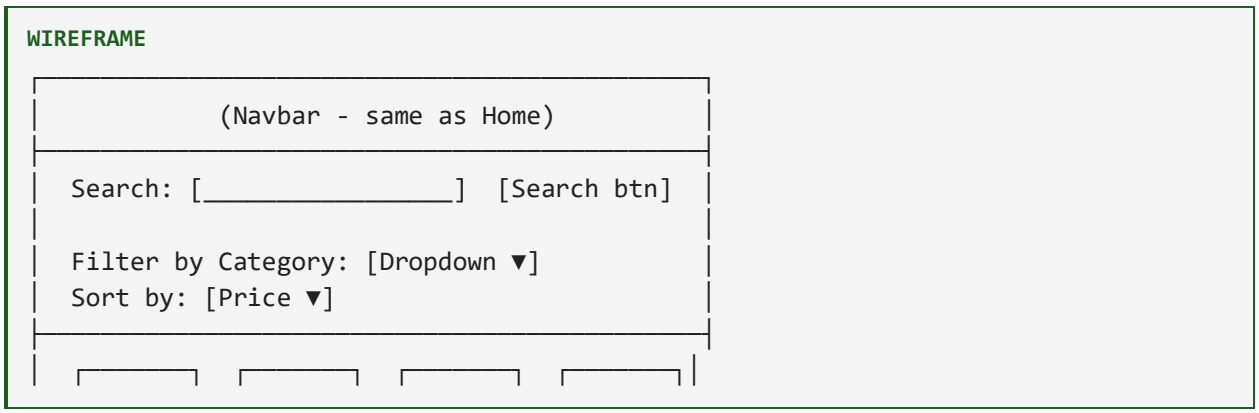
Wireframes - Sketching the UI

Before pixels, draw boxes. A wireframe is a rough sketch of where things go - no colors, no fonts, just structure.

Wireframe: Home Page



Wireframe: Books Listing Page



```
|  | Cover |  | Cover |  | Cover |  | Cover ||  ← Grid of
|  | Title |  | Title |  | Title |  | Title ||    book cards
|  | Author|  | Author|  | Author|  | Author||
|  | ₹299  |  | ₹349  |  | ₹499  |  | ₹199  ||
|  | [Add] |  | [Add] |  | [Add] |  | [Add] ||
|  └───────┘  └───────┘  └───────┘  └───────┘|
|                                            |
|  ┌───────┐  ┌───────┐  ┌───────┐  ┌───────┐|
|  |   ...repeating...                       ||
└──────────────────────────────────────────────┘
```

Industry Tool

Real teams use Figma to design wireframes. For this course, hand sketches or Whimsical.com (free) work great. The point isn't fancy tools - it's THINKING before coding.

Color Palette - Brand Identity

```css
Primary:     #5B3A29  (Coffee Brown)    - Like an old leather book
Accent:      #F4A261  (Warm Orange)     - Highlights, CTAs
Background:  #FFF8F0  (Cream White)     - Page background
Dark Text:   #2C2C2C  (Almost Black)    - Body text
Muted Text:  #6B6B6B  (Gray)            - Secondary text
Success:     #2E7D32  (Green)           - Success messages
Error:       #C62828  (Red)             - Errors, warnings
```

These 7 colors will be used across the entire website. Consistency builds trust.

Typography

```css
Headings:  'Playfair Display'  (serif - elegant, bookish)
Body:      'Inter'             (sans-serif - modern, readable)
Code/Mono: 'Courier New'       (monospace - if showing code)
```

Two fonts max - one for headings, one for body. Three or more creates visual noise.

Today's Tasks for Students

- Draw your own wireframes for all 6 pages on paper or Whimsical.
- Pick a different color palette (3-5 colors) - explore coolors.co.
- Choose 2 fonts from fonts.google.com.
- Write a one-paragraph project vision (your unique angle).

PHASE 2: PROJECT SETUP & FOLDER STRUCTURE

Why Folder Structure Matters

Imagine opening a college student's project: 50 files all in one folder, named randomly like 'newpage.html', 'new1.html', 'aaa.css'. Now imagine opening a company project: clean folders, logical names, easy to find anything. The difference is professionalism.

Today we set up BookHive's folder structure the way professionals do.

Tools Setup

1. VS Code (Your Code Editor)

- Download from code.visualstudio.com if you haven't already.
- Install extensions: Live Server, Prettier, Auto Rename Tag, Live Preview.

2. Browser

- Use Chrome with DevTools (F12) - this is your debugging toolbox.

3. Git & GitHub

- Install Git from git-scm.com.
- Create a GitHub account at github.com if you don't have one.

Creating the Project Folder

```
TERMINAL
# Open terminal/command prompt
# Navigate to where you keep projects (Desktop is fine for learning)

cd Desktop

# Create the main project folder
mkdir bookhive

# Go inside it
cd bookhive

# Open this folder in VS Code
code .
```

The Folder Structure - Industry Standard

```
STRUCTURE
bookhive/
```

```
│
├── index.html              ← Home page (entry point)
├── books.html              ← Books listing page
├── book.html               ← Single book detail page
├── cart.html               ← Shopping cart page
├── login.html              ← Login page
├── signup.html             ← Signup page
├── about.html              ← About us page
├── contact.html            ← Contact page
│
├── css/                    ← All CSS files
│   ├── style.css           ← Main styles (used everywhere)
│   ├── home.css            ← Home page specific
│   ├── books.css           ← Books page specific
│   └── responsive.css      ← Media queries
│
├── js/                     ← All JavaScript files
│   ├── main.js             ← Common scripts (navbar, cart count)
│   ├── books.js            ← Book data (array of book objects)
│   ├── cart.js             ← Cart functionality
│   └── search.js           ← Search and filter
│
├── images/                 ← All images
│   ├── logo.png
│   ├── hero-bg.jpg
│   └── books/              ← Book cover images
│       ├── book1.jpg
│       └── book2.jpg
│
├── assets/                 ← Other resources
│   └── icons/              ← Icon files (SVG, PNG)
│
├── README.md               ← Project description
└── .gitignore              ← Files to ignore in Git
```

Teaching Moment: Notice the pattern - similar files grouped together. CSS files in css/, JS in js/, images in images/. This is what every professional project looks like. Random files in root = amateur. Organized folders = professional.

Create the Files - Step by Step

Step 1: Create all HTML files

In VS Code, right-click the 'bookhive' folder in the sidebar. Choose 'New File'. Create these one by one:

- index.html
- books.html
- book.html
- cart.html

- login.html
- signup.html
- about.html
- contact.html

Step 2: Create the folders

Right-click 'bookhive' → New Folder. Create:

- css
- js
- images
- assets

Step 3: Create CSS files inside css/

- style.css
- home.css
- books.css
- responsive.css

Step 4: Create JS files inside js/

- main.js
- books.js
- cart.js
- search.js

Initialize Git Repository

```
TERMINAL
# Inside bookhive folder
git init

# Create a .gitignore file with this content:
.DS_Store
.vscode/
node_modules/
*.log

# Make first commit
git add .
git commit -m "Initial project setup - empty file structure"

# Create a repo on GitHub.com called 'bookhive'
# Then connect:
git remote add origin https://github.com/YOUR-USERNAME/bookhive.git
git branch -M main
```

```
git push -u origin main
```

Create a Professional README.md

```
MARKDOWN
# 📚 BookHive - Online Bookstore

A modern, responsive online bookstore showcasing Indian authors and bestsellers.

## 🌟 Features

- Browse and search books by category
- Detailed book pages with descriptions
- Shopping cart with quantity controls
- User signup and login pages
- Fully responsive design (mobile, tablet, desktop)

## 🛠 Built With

- **HTML5** - Semantic markup
- **CSS3** - Custom styling, Flexbox, Grid, animations
- **JavaScript (ES6+)** - Interactivity, localStorage, DOM manipulation

## 🚀 Live Demo

Coming soon!

## 📁 Project Structure

```
bookhive/
├── index.html
├── css/
├── js/
└── images/
```

👤 Author

Built by [Your Name] as part of the MERN Stack journey.
```

Today's Tasks for Students

- Create the entire folder structure exactly as above.
- Create the GitHub repo and push the empty structure.

- Write the README.md with your own name.
- Commit message: 'Initial setup - folder structure ready'.

> **Pro Tip**
>
> Commit often, even when files are empty. Each commit is a checkpoint. If something breaks later, you can always come back to a working state.

PHASE 3: HTML SKELETON - BUILDING THE HOME PAGE

Topic Introduction: HTML5 Boilerplate & Semantic Tags

Now we build! Today we add HTML to the empty files. But instead of teaching HTML tags from a list, we'll INTRODUCE EACH TAG WHEN WE NEED IT. This is how real learning happens - in context, not abstractly.

Teaching Moment: Demo this on the projector: open index.html in browser - it's blank. Then write HTML and refresh - structure appears. Magic. Now they're curious.

Step 1: The HTML5 Boilerplate

Open index.html. Type ! and press Tab (Emmet shortcut). VS Code auto-generates the boilerplate. Let me explain each line.

```HTML
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>

</body>
</html>
```

Teaching Each Line

- **<!DOCTYPE html>:** Tells the browser 'this is HTML5'. ALWAYS the first line.
- **<html lang="en">:** The root tag. lang='en' helps Google understand it's English content.
- **<meta charset="UTF-8">:** Character encoding. Without it, special characters like ₹ might break.
- **<meta name="viewport">:** Makes site mobile-friendly. Without this, sites look tiny on phones.
- **<title>:** Shows in the browser tab and Google search results.
- **<body>:** Everything VISIBLE on the page goes here.

Step 2: Customize the Boilerplate for BookHive

```HTML
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <!-- SEO meta tags -->
    <title>BookHive - India's Favorite Online Bookstore</title>
    <meta name="description" content="Buy books online at BookHive. Indian
authors, bestsellers, fiction, education books. Free delivery across India.">
    <meta name="keywords" content="books, online bookstore, indian authors, buy
books online">

    <!-- Open Graph for social media -->
    <meta property="og:title" content="BookHive - Online Bookstore">
    <meta property="og:description" content="India's favorite place to buy books
online">
    <meta property="og:image" content="images/og-banner.jpg">

    <!-- Favicon -->
    <link rel="icon" href="images/favicon.ico">
</head>
<body>

</body>
</html>
```

Teaching Moment: These meta tags are why Google can find websites. Skip them, and even great websites stay invisible. Recruiters notice when you include them.

Step 3: Building the Page Structure with Semantic Tags

Now we add content using SEMANTIC HTML5 tags. Each tag has MEANING - it tells search engines and screen readers what each part is.

The 5 Semantic Tags We'll Use

Tag	Used For
<header>	Top of page - logo and navigation
<nav>	Navigation links menu
<main>	The main content area (ONE per page)
<section>	A thematic group of content with a heading
<footer>	Bottom of page - copyright, links

Step 4: Write the Home Page HTML

Add this inside the <body> tag of index.html:

```HTML
```

```
<body>

    <!-- HEADER WITH NAVIGATION -->
    <header>
        <h1>📚 BookHive</h1>

        <nav>
            <a href="index.html">Home</a>
            <a href="books.html">Books</a>
            <a href="about.html">About</a>
            <a href="contact.html">Contact</a>
            <a href="cart.html">🛒 Cart (0)</a>
            <a href="login.html">Login</a>
        </nav>
    </header>

    <!-- MAIN CONTENT -->
    <main>

        <!-- HERO SECTION -->
        <section class="hero">
            <h2>Welcome to BookHive</h2>
            <p>India's favorite place to discover great books</p>
            <a href="books.html" class="btn">Browse Books</a>
        </section>

        <!-- FEATURED BOOKS SECTION -->
        <section class="featured">
            <h2>Featured Books</h2>

            <article class="book-card">
                <img src="images/books/wings-of-fire.jpg" alt="Wings of Fire book
cover">
                <h3>Wings of Fire</h3>
                <p>by A.P.J. Abdul Kalam</p>
                <p class="price">₹299</p>
                <button>Add to Cart</button>
            </article>

            <article class="book-card">
                <img src="images/books/the-alchemist.jpg" alt="The Alchemist book
cover">
                <h3>The Alchemist</h3>
                <p>by Paulo Coelho</p>
                <p class="price">₹349</p>
                <button>Add to Cart</button>
            </article>
```

```
            <article class="book-card">
                <img src="images/books/atomic-habits.jpg" alt="Atomic Habits book
cover">
                <h3>Atomic Habits</h3>
                <p>by James Clear</p>
                <p class="price">₹499</p>
                <button>Add to Cart</button>
            </article>

            <article class="book-card">
                <img src="images/books/sapiens.jpg" alt="Sapiens book cover">
                <h3>Sapiens</h3>
                <p>by Yuval Noah Harari</p>
                <p class="price">₹599</p>
                <button>Add to Cart</button>
            </article>
        </section>

        <!-- CATEGORIES SECTION -->
        <section class="categories">
            <h2>Browse by Category</h2>

            <div class="category">
                <h3>📖 Fiction</h3>
                <p>Novels, short stories, fantasy</p>
            </div>

            <div class="category">
                <h3>🎓 Education</h3>
                <p>Textbooks, study guides</p>
            </div>

            <div class="category">
                <h3>👤 Biographies</h3>
                <p>Life stories of remarkable people</p>
            </div>

            <div class="category">
                <h3>💼 Business</h3>
                <p>Entrepreneurship, leadership</p>
            </div>
        </section>

    </main>

    <!-- FOOTER -->
```

```
    <footer>
        <p>&copy; 2026 BookHive. All rights reserved.</p>
        <p>
            <a href="about.html">About</a> |
            <a href="contact.html">Contact</a> |
            <a href="#">Privacy Policy</a>
        </p>
    </footer>

</body>
```

Save the file (Ctrl+S). Right-click in the editor → 'Open with Live Server'. The page opens in your browser.

Teaching Moment: Right now it looks like a 1995 website - all text, no style. That's INTENTIONAL. Students see structure FIRST, then we add style in Phase 6. Many beginners style and structure at the same time and get confused.

Topics Taught in This Phase

- HTML5 boilerplate - the standard starting point.
- Meta tags - charset, viewport, SEO, Open Graph.
- Semantic HTML5 tags: header, nav, main, section, article, footer.
- Heading hierarchy (h1, h2, h3).
- Anchor tags <a> with href.
- Image tags with alt attribute.
- Buttons (we'll make them work in Phase 11).
- Class attribute (used by CSS in Phase 6).

Today's Tasks for Students

- Type the entire home page HTML (don't copy-paste - typing builds muscle memory).
- Add at least 6 featured books (not just 4).
- Add 6 categories instead of 4.
- Customize the welcome message to your style.
- Commit: 'Phase 3 - Home page HTML structure complete'.

Common Mistakes

- Forgetting closing tags. <h2>Hello (without </h2>) breaks the layout.
- Using <h1> multiple times. Use it ONCE per page (the main title).
- Missing alt attribute on images. Add it for SEO and accessibility.
- Using
 for spacing. That's CSS's job, not HTML's.

PHASE 4: HTML FORMS - LOGIN & SIGNUP PAGES

Topic Introduction: HTML Forms

Forms are how users TALK to websites. Every signup, login, search, contact - all forms. Today we build BookHive's login and signup pages. As we build, students learn every essential form element.

Teaching Moment: Ask students: 'How many forms have you filled today?' Gmail login, Instagram search, Swiggy address, Razorpay payment, college portal - we use forms constantly. Today they learn to BUILD them.

Build the Login Page (login.html)

```HTML
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login - BookHive</title>
</head>
<body>

    <!-- Same header as home page -->
    <header>
        <h1> BookHive</h1>
        <nav>
            <a href="index.html">Home</a>
            <a href="books.html">Books</a>
            <a href="about.html">About</a>
            <a href="contact.html">Contact</a>
            <a href="cart.html"> Cart (0)</a>
            <a href="login.html">Login</a>
        </nav>
    </header>

    <main>
        <section class="form-section">
            <h2>Welcome Back!</h2>
            <p>Login to your BookHive account</p>

            <form action="#" method="POST" class="login-form">

                <!-- Email field -->
                <div class="form-group">
                    <label for="email">Email Address</label>
                    <input
```

```
                    type="email"
                    id="email"
                    name="email"
                    placeholder="you@example.com"
                    required
                >
            </div>

            <!-- Password field -->
            <div class="form-group">
                <label for="password">Password</label>
                <input
                    type="password"
                    id="password"
                    name="password"
                    placeholder="Enter your password"
                    minlength="6"
                    required
                >
            </div>

            <!-- Remember me checkbox -->
            <div class="form-group checkbox-group">
                <input type="checkbox" id="remember" name="remember">
                <label for="remember">Remember me</label>
            </div>

            <!-- Submit button -->
            <button type="submit" class="btn-primary">Login</button>

            <!-- Forgot password link -->
            <p class="form-link">
                <a href="#">Forgot password?</a>
            </p>

            <!-- Link to signup -->
            <p class="form-link">
                Don't have an account?
                <a href="signup.html">Sign up here</a>
            </p>
        </form>
    </section>
</main>

<footer>
    <p>&copy; 2026 BookHive. All rights reserved.</p>
</footer>
```

```
</body>
</html>
```

Teaching Each Form Element

The <form> Tag

Wraps all input fields together. Two important attributes:

- **action:** Where to send the data when submitted. In real apps, this is a server URL. For now, '#' (we'll handle with JS later).
- **method:** How to send it. POST for forms with passwords or large data. GET for search forms.

The <label> Tag

The text that tells the user what an input is for. Always connect labels to inputs with 'for' and 'id' that match. This helps blind users using screen readers.

Input Types Used

- type="email" - auto-validates email format
- type="password" - hides characters as user types
- type="checkbox" - small box that can be checked/unchecked

Input Attributes Used

- placeholder - hint text inside the input
- required - field must be filled
- minlength - minimum character count
- id - unique identifier (for label and CSS)
- name - what the form sends to the server

Build the Signup Page (signup.html)

Signup is a longer form. More fields = more practice with form elements.

```HTML
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sign Up - BookHive</title>
</head>
<body>

    <header>
        <h1>📖 BookHive</h1>
        <nav>
            <a href="index.html">Home</a>
```

```
            <a href="books.html">Books</a>
            <a href="about.html">About</a>
            <a href="contact.html">Contact</a>
            <a href="cart.html">🛒 Cart (0)</a>
            <a href="login.html">Login</a>
        </nav>
    </header>

    <main>
        <section class="form-section">
            <h2>Create Your Account</h2>
            <p>Join thousands of book lovers on BookHive</p>

            <form action="#" method="POST" class="signup-form">

                <div class="form-group">
                    <label for="fullname">Full Name *</label>
                    <input
                        type="text"
                        id="fullname"
                        name="fullname"
                        placeholder="Enter your full name"
                        required
                        minlength="3"
                    >
                </div>

                <div class="form-group">
                    <label for="email">Email Address *</label>
                    <input
                        type="email"
                        id="email"
                        name="email"
                        placeholder="you@example.com"
                        required
                    >
                </div>

                <div class="form-group">
                    <label for="phone">Phone Number</label>
                    <input
                        type="tel"
                        id="phone"
                        name="phone"
                        placeholder="10-digit mobile number"
                        pattern="[0-9]{10}"
                    >
                </div>
```

```
            <div class="form-group">
                <label for="password">Password *</label>
                <input
                    type="password"
                    id="password"
                    name="password"
                    placeholder="At least 8 characters"
                    minlength="8"
                    required
                >
                <small>Must be at least 8 characters</small>
            </div>

            <div class="form-group">
                <label for="confirm-password">Confirm Password *</label>
                <input
                    type="password"
                    id="confirm-password"
                    name="confirm-password"
                    placeholder="Re-enter your password"
                    required
                >
            </div>

            <div class="form-group">
                <label for="city">City</label>
                <select id="city" name="city">
                    <option value="">-- Select your city --</option>
                    <option value="bengaluru">Bengaluru</option>
                    <option value="mumbai">Mumbai</option>
                    <option value="delhi">Delhi</option>
                    <option value="hyderabad">Hyderabad</option>
                    <option value="chennai">Chennai</option>
                    <option value="pune">Pune</option>
                    <option value="kolkata">Kolkata</option>
                </select>
            </div>

            <div class="form-group">
                <label for="dob">Date of Birth</label>
                <input type="date" id="dob" name="dob">
            </div>

            <div class="form-group">
                <p>I'm interested in: (check all that apply)</p>
                <div class="checkbox-grid">
```

```
                        <label><input type="checkbox" name="interests"
value="fiction"> Fiction</label>
                        <label><input type="checkbox" name="interests"
value="education"> Education</label>
                        <label><input type="checkbox" name="interests"
value="biography"> Biographies</label>
                        <label><input type="checkbox" name="interests"
value="business"> Business</label>
                        <label><input type="checkbox" name="interests"
value="children"> Children's</label>
                        <label><input type="checkbox" name="interests"
value="comics"> Comics</label>
                    </div>
                </div>

                <div class="form-group checkbox-group">
                    <input type="checkbox" id="terms" name="terms" required>
                    <label for="terms">I agree to the Terms and Conditions
*</label>
                </div>

                <div class="form-group checkbox-group">
                    <input type="checkbox" id="newsletter" name="newsletter">
                    <label for="newsletter">Subscribe to weekly book
recommendations</label>
                </div>

                <button type="submit" class="btn-primary">Create Account</button>

                <p class="form-link">
                    Already have an account?
                    <a href="login.html">Login here</a>
                </p>
            </form>
        </section>
    </main>

    <footer>
        <p>&copy; 2026 BookHive. All rights reserved.</p>
    </footer>

</body>
</html>
```

New Form Elements Introduced

<select> and <option> - Dropdowns

For when users pick from a list. Common use: country, state, city. The 'value' attribute is what gets sent to the server. The text between tags is what user sees.

Multiple Checkboxes with Same Name

All 'interests' checkboxes share name='interests'. When user submits, the server gets an ARRAY of selected values. That's how 'select all that apply' forms work.

type="date" - Date Picker

Browser shows a native date picker UI - no JavaScript needed. Mobile shows phone's date picker. Desktop shows a calendar.

type="tel" with pattern

Tel input shows numeric keyboard on mobile. The pattern attribute uses regex - '[0-9]{10}' means 'exactly 10 digits'.

Build the Contact Page (contact.html)

```HTML
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Contact Us - BookHive</title>
</head>
<body>

    <header>
        <h1> BookHive</h1>
        <nav>
            <a href="index.html">Home</a>
            <a href="books.html">Books</a>
            <a href="about.html">About</a>
            <a href="contact.html">Contact</a>
            <a href="cart.html">🛒 Cart (0)</a>
            <a href="login.html">Login</a>
        </nav>
    </header>

    <main>
        <section class="contact-section">
            <h2>Get in Touch</h2>
            <p>Have a question or feedback? We'd love to hear from you.</p>

            <form action="#" method="POST" class="contact-form">

                <div class="form-group">
```

```
                <label for="name">Your Name *</label>
                <input type="text" id="name" name="name" required>
            </div>

            <div class="form-group">
                <label for="email">Email *</label>
                <input type="email" id="email" name="email" required>
            </div>

            <div class="form-group">
                <label for="subject">Subject</label>
                <select id="subject" name="subject">
                    <option value="general">General Inquiry</option>
                    <option value="order">Order Issue</option>
                    <option value="suggestion">Book Suggestion</option>
                    <option value="partnership">Partnership</option>
                </select>
            </div>

            <div class="form-group">
                <label for="message">Your Message *</label>
                <textarea
                    id="message"
                    name="message"
                    rows="6"
                    placeholder="Tell us what's on your mind..."
                    required
                ></textarea>
            </div>

            <button type="submit" class="btn-primary">Send Message</button>
        </form>
    </section>

    <section class="contact-info">
        <h3>Other Ways to Reach Us</h3>
        <p>📧 Email: <a
href="mailto:hello@bookhive.in">hello@bookhive.in</a></p>
        <p>📞 Phone: <a href="tel:+918012345678">+91 80-1234-5678</a></p>
        <p>📍 Address: 123 MG Road, Bengaluru, Karnataka 560001</p>
    </section>
</main>

<footer>
    <p>&copy; 2026 BookHive. All rights reserved.</p>
</footer>

</body>
```

```
</html>
```

New Element: <textarea>

For multi-line text input. Unlike <input>, it has both opening and closing tags. The 'rows' attribute sets the visible height.

Special href Values

- mailto:hello@example.com - opens email client
- tel:+919876543210 - opens phone dialer on mobile

Topics Taught in This Phase

- <form> tag with action and method attributes
- <label> connected to <input> via for/id
- Input types: text, email, password, tel, date, checkbox
- Input attributes: placeholder, required, minlength, pattern, name
- <select> and <option> for dropdowns
- <textarea> for multi-line input
- Multiple checkboxes with same name (arrays)
- Special hrefs: mailto:, tel:
- HTML5 built-in validation (required, minlength, pattern)

Today's Tasks for Students

- Type all three forms (login, signup, contact) from scratch.
- Open in browser, try to submit empty - see HTML5 validation in action.
- Add another form field to signup: 'Profession' (dropdown: Student, Working, Other).
- Try every input type yourself - color, range, number, search.
- Commit: 'Phase 4 - Login, signup, contact forms complete'.

PHASE 5: HTML LISTS, TABLES & MEDIA - BOOK LISTINGS

Topic Introduction: Structuring Data

Today we build the books listing page and cart page. These pages show LOTS of data in organized ways. Perfect time to teach HTML lists, tables, and media elements.

Build the Books Listing Page (books.html)

```HTML
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>All Books - BookHive</title>
</head>
<body>

    <header>
        <h1>📚 BookHive</h1>
        <nav>
            <a href="index.html">Home</a>
            <a href="books.html">Books</a>
            <a href="about.html">About</a>
            <a href="contact.html">Contact</a>
            <a href="cart.html">🛒 Cart (0)</a>
            <a href="login.html">Login</a>
        </nav>
    </header>

    <main>

        <!-- PAGE TITLE -->
        <section class="page-header">
            <h2>All Books</h2>
            <p>Browse our complete collection of 2,000+ books</p>
        </section>

        <!-- SEARCH AND FILTERS -->
        <section class="filters">
            <div class="search-box">
                <input
                    type="search"
                    id="search-input"
                    placeholder="Search books, authors, or genres..."
```

```
                >
                <button type="button">Search</button>
            </div>

            <div class="filter-controls">
                <label for="category-filter">Category:</label>
                <select id="category-filter">
                    <option value="all">All Categories</option>
                    <option value="fiction">Fiction</option>
                    <option value="education">Education</option>
                    <option value="biography">Biographies</option>
                    <option value="business">Business</option>
                </select>

                <label for="sort">Sort by:</label>
                <select id="sort">
                    <option value="default">Default</option>
                    <option value="price-low">Price: Low to High</option>
                    <option value="price-high">Price: High to Low</option>
                    <option value="rating">Rating</option>
                </select>
            </div>
        </section>

        <!-- POPULAR CATEGORIES (using unordered list) -->
        <section class="categories-quick">
            <h3>Popular Categories</h3>
            <ul class="category-list">
                <li><a href="#fiction">Fiction</a></li>
                <li><a href="#nonfiction">Non-Fiction</a></li>
                <li><a href="#biography">Biographies</a></li>
                <li><a href="#business">Business</a></li>
                <li><a href="#children">Children</a></li>
                <li><a href="#comics">Comics</a></li>
            </ul>
        </section>

        <!-- BOOKS GRID -->
        <section class="books-grid" id="books-container">

            <article class="book-card">
                <img src="images/books/wings-of-fire.jpg" alt="Wings of Fire"
loading="lazy">
                <h3>Wings of Fire</h3>
                <p class="author">A.P.J. Abdul Kalam</p>
                <p class="rating">⭐ 4.8 (2,341 reviews)</p>
                <p class="price">₹299</p>
                <button class="add-to-cart" data-id="1">Add to Cart</button>
```

```
            </article>

            <article class="book-card">
                <img src="images/books/the-alchemist.jpg" alt="The Alchemist"
loading="lazy">
                <h3>The Alchemist</h3>
                <p class="author">Paulo Coelho</p>
                <p class="rating">★ 4.7 (5,012 reviews)</p>
                <p class="price">₹349</p>
                <button class="add-to-cart" data-id="2">Add to Cart</button>
            </article>

            <article class="book-card">
                <img src="images/books/atomic-habits.jpg" alt="Atomic Habits"
loading="lazy">
                <h3>Atomic Habits</h3>
                <p class="author">James Clear</p>
                <p class="rating">★ 4.9 (8,420 reviews)</p>
                <p class="price">₹499</p>
                <button class="add-to-cart" data-id="3">Add to Cart</button>
            </article>

            <!-- Add 9-12 more book cards following the same pattern -->

        </section>

    </main>

    <footer>
        <p>&copy; 2026 BookHive. All rights reserved.</p>
    </footer>

</body>
</html>
```

Teaching New Concepts in This Page

Unordered Lists (ul, li)

Used for the category navigation. creates an unordered list (bullets). Each <li> is a list item. Lists are often used for menus - what looks like a horizontal menu is often a styled <ul>.

type="search"

Like text input, but shows an 'x' clear button. Mobile shows search-optimized keyboard. Tiny detail, but professional.

loading="lazy"

Tells browser to load images ONLY when they're about to appear on screen. Massively improves page speed when you have many images. Add it to every below-the-fold image.

data-* Attributes

Custom attributes that start with 'data-'. We use data-id="1" to store the book's ID. JavaScript can read these later when user clicks 'Add to Cart'. This is HOW we'll connect HTML to JS in Phase 11.

Teaching Moment: data-* attributes are the bridge between HTML and JavaScript. Every dynamic website uses them. Razorpay product cards have data-product-id. Swiggy menu items have data-item-id. Now your students know this trick.

Build the Cart Page (cart.html) - With a Table

```HTML
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Your Cart - BookHive</title>
</head>
<body>

    <header>
        <h1> BookHive</h1>
        <nav>
            <a href="index.html">Home</a>
            <a href="books.html">Books</a>
            <a href="about.html">About</a>
            <a href="contact.html">Contact</a>
            <a href="cart.html">Cart (3)</a>
            <a href="login.html">Login</a>
        </nav>
    </header>

    <main>
        <section class="cart-section">
            <h2>Your Shopping Cart</h2>

            <!-- Table for cart items -->
            <table class="cart-table">
                <thead>
                    <tr>
                        <th>Book</th>
                        <th>Title</th>
                        <th>Price</th>
                        <th>Quantity</th>
                        <th>Subtotal</th>
```

```
                        <th>Remove</th>
                    </tr>
                </thead>
                <tbody id="cart-items">
                    <tr>
                        <td>
                            <img src="images/books/wings-of-fire.jpg" alt=""
width="60">
                        </td>
                        <td>
                            <strong>Wings of Fire</strong><br>
                            <small>A.P.J. Abdul Kalam</small>
                        </td>
                        <td>₹299</td>
                        <td>
                            <input type="number" value="1" min="1" max="10"
class="qty-input">
                        </td>
                        <td>₹299</td>
                        <td>
                            <button class="remove-btn">✕/button>
                        </td>
                    </tr>
                    <tr>
                        <td>
                            <img src="images/books/the-alchemist.jpg" alt=""
width="60">
                        </td>
                        <td>
                            <strong>The Alchemist</strong><br>
                            <small>Paulo Coelho</small>
                        </td>
                        <td>₹349</td>
                        <td>
                            <input type="number" value="2" min="1" max="10"
class="qty-input">
                        </td>
                        <td>₹698</td>
                        <td>
                            <button class="remove-btn">✕/button>
                        </td>
                    </tr>
                </tbody>
                <tfoot>
                    <tr>
                        <td colspan="4"><strong>Total:</strong></td>
                        <td colspan="2"><strong id="cart-total">₹997</strong></td>
                    </tr>
                </tfoot>
```

```
            </table>

            <!-- Order summary -->
            <section class="order-summary">
                <h3>Order Summary</h3>
                <dl>
                    <dt>Subtotal:</dt>
                    <dd>₹997</dd>

                    <dt>Shipping:</dt>
                    <dd>FREE</dd>

                    <dt>Tax (5% GST):</dt>
                    <dd>₹49.85</dd>

                    <dt><strong>Grand Total:</strong></dt>
                    <dd><strong>₹1,046.85</strong></dd>
                </dl>

                <a href="#" class="btn-primary">Proceed to Checkout</a>
                <a href="books.html" class="btn-secondary">Continue Shopping</a>
            </section>
        </section>
    </main>

    <footer>
        <p>&copy; 2026 BookHive. All rights reserved.</p>
    </footer>

</body>
</html>
```

Teaching Tables in HTML

Table Anatomy

- **<table>:** The whole table container.
- **<thead>:** Table header section (column titles).
- **<tbody>:** Table body (the actual rows of data).
- **<tfoot>:** Table footer (totals, summaries).
- **<tr>:** Table row.
- **<th>:** Header cell (bold, centered by default).
- **<td>:** Data cell (the actual content).

colspan and rowspan

colspan="4" means 'this cell spans 4 columns'. Look at the footer - the 'Total:' label spans 4 columns, while the total amount spans 2. Used for merged cells.

IMPORTANT Rule About Tables

Use tables ONLY for tabular data (data that naturally has rows and columns). The cart is perfect - it's a list of items with the same attributes. NEVER use tables for page layouts (that's CSS's job). 20 years ago developers used tables for layouts - that's now BAD practice.

Definition Lists (dl, dt, dd)

In the order summary, we use dl/dt/dd - the 'definition list'. dt is the term (Subtotal), dd is the description (₹997). Less common but semantically perfect for key-value pairs.

Build the Single Book Page (book.html)

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Wings of Fire - BookHive</title>
</head>
<body>

    <header>
        <h1> BookHive</h1>
        <nav>
            <a href="index.html">Home</a>
            <a href="books.html">Books</a>
            <a href="about.html">About</a>
            <a href="contact.html">Contact</a>
            <a href="cart.html">🛒 Cart (0)</a>
            <a href="login.html">Login</a>
        </nav>
    </header>

    <main>
        <!-- Breadcrumb navigation using a list -->
        <nav class="breadcrumb">
            <ol>
                <li><a href="index.html">Home</a></li>
                <li><a href="books.html">Books</a></li>
                <li>Wings of Fire</li>
            </ol>
        </nav>

        <article class="book-detail">

            <div class="book-image">
```

```
                <img src="images/books/wings-of-fire.jpg" alt="Wings of Fire book
cover">
            </div>

            <div class="book-info">
                <h2>Wings of Fire</h2>
                <p class="author">by <strong>A.P.J. Abdul Kalam</strong></p>

                <div class="rating">
                    ★★★★★ 4.8 out of 5 (2,341 reviews)
                </div>

                <p class="price">₹299 <small><s>₹499</s> (40% off)</small></p>

                <p class="description">
                    Wings of Fire is the autobiography of A.P.J. Abdul Kalam,
India's
                    11th president and a renowned aerospace scientist. The book
traces
                    his journey from a small town in Tamil Nadu to becoming one of
                    India's most beloved figures.
                </p>

                <!-- Book details using a description list -->
                <h3>Book Details</h3>
                <dl class="book-details">
                    <dt>Publisher:</dt>
                    <dd>Universities Press</dd>

                    <dt>Language:</dt>
                    <dd>English</dd>

                    <dt>Pages:</dt>
                    <dd>180</dd>

                    <dt>ISBN:</dt>
                    <dd>9788173711466</dd>

                    <dt>Format:</dt>
                    <dd>Paperback</dd>
                </dl>

                <!-- Quantity and add to cart -->
                <div class="purchase-actions">
                    <label for="quantity">Quantity:</label>
                    <input type="number" id="quantity" value="1" min="1" max="10">
                    <button class="btn-primary">Add to Cart</button>
```

```
                    <button class="btn-secondary">Buy Now</button>
                </div>
            </div>
        </article>

        <!-- Reviews section -->
        <section class="reviews">
            <h3>Customer Reviews</h3>

            <article class="review">
                <h4>Inspirational and motivating</h4>
                <p class="reviewer">By <strong>Rahul S.</strong> | ★★★★★ | 15 Mar
2025</p>
                <p>This book changed my perspective on hard work and dedication.
Kalam's
                journey from humble beginnings is truly inspirational.</p>
            </article>

            <article class="review">
                <h4>A must-read for every Indian</h4>
                <p class="reviewer">By <strong>Priya M.</strong> | ★★★★★ | 2 Apr
2025</p>
                <p>Couldn't put it down. Beautifully written and deeply moving.
                Recommended to everyone.</p>
            </article>
        </section>

    </main>

    <footer>
        <p>&copy; 2026 BookHive. All rights reserved.</p>
    </footer>

</body>
</html>
```

New Concept: Breadcrumb with Ordered List

Breadcrumb shows where you are in the site hierarchy. We use <ol> (ordered list) because the order matters - Home > Books > This Book. CSS will style it as inline text with separators.

New Concept: <s> Tag (Strikethrough)

Crosses out text. Used for showing original price (struck out) next to discounted price. Like '₹499' crossed out with '₹299' next to it.

Topics Taught in This Phase

- Unordered lists (<ul>, <li>)

- Ordered lists (<ol>, <li>) for breadcrumbs
- Definition lists (<dl>, <dt>, <dd>)
- Tables: <table>, <thead>, <tbody>, <tfoot>, <tr>, <th>, <td>
- colspan and rowspan attributes
- type="search" and type="number"
- loading="lazy" for image performance
- data-* custom attributes (bridge to JavaScript)
- <s> tag for strikethrough text
- <article> for self-contained content (book card, review)

Today's Tasks for Students

- Type the entire books.html with 12+ book cards.
- Type the entire cart.html with proper table structure.
- Type the entire book.html with all sections.
- Add at least 3 customer reviews to book.html.
- Make sure ALL book images have alt attributes.
- Open all pages in browser, verify navigation works between pages.
- Commit: 'Phase 5 - Books listing, cart, single book pages complete'.

HTML Phase Complete!

Students have now built 8 HTML pages with complete structure. The site is functionally COMPLETE in terms of structure - users could fill forms, navigate, click links. But it looks like a 1995 website. Phase 6 onwards: CSS magic begins.

PHASE 6: CSS FOUNDATIONS - STYLING THE HOME PAGE

Topic Introduction: From Plain to Polished

Today, the website transforms. Students open Live Server before adding CSS - boring. They add the CSS file - WOW. This 'before and after' moment is powerful for engagement.

Teaching Moment: Demo the transformation in real-time. Type ONE CSS rule, refresh, show change. Then another, refresh. By the end of the lesson, students physically see the website come alive.

Step 1: Link CSS to HTML

Open index.html. Add this inside the <head> tag:

```HTML
<link rel="stylesheet" href="css/style.css">
```

Do this for ALL HTML pages. Every page links to style.css. One change in CSS = all 8 pages update.

Step 2: The CSS Reset

Every browser has default styles - different margins, paddings, fonts. To start clean, we 'reset' them. Open css/style.css and add:

```CSS
/* ============================
   CSS RESET - Start Clean
   ============================ */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

html {
    font-size: 16px;
}

body {
    font-family: 'Inter', Arial, sans-serif;
    color: #2C2C2C;
    background-color: #FFF8F0;
    line-height: 1.6;
}
```

Teaching Each Rule

The Universal Selector *

Targets EVERY element on the page. We're saying: 'For every element, set margin to 0, padding to 0, and use border-box.' This gives us a consistent starting point.

box-sizing: border-box

Without this: width: 100px + padding: 20px = ACTUAL 140px wide (padding adds OUTSIDE the width). Confusing.

With border-box: width: 100px INCLUDES padding and border. Total is always 100px. Predictable.

> **Memorize This**
>
> EVERY professional CSS file starts with * { box-sizing: border-box; }. Tailwind does it. Bootstrap does it. Every framework. Add it to every project.

Step 3: CSS Variables - Smart Color Management

Instead of writing '#5B3A29' 50 times, define it once as a variable:

```css
/* ============================
   CSS VARIABLES - Brand Colors
   ============================ */
:root {
    /* Colors */
    --primary: #5B3A29;
    --accent: #F4A261;
    --bg: #FFF8F0;
    --text-dark: #2C2C2C;
    --text-muted: #6B6B6B;
    --success: #2E7D32;
    --error: #C62828;
    --white: #FFFFFF;
    --light-gray: #F5F5F5;

    /* Spacing */
    --space-xs: 0.5rem;   /* 8px */
    --space-sm: 1rem;     /* 16px */
    --space-md: 1.5rem;   /* 24px */
    --space-lg: 2rem;     /* 32px */
    --space-xl: 3rem;     /* 48px */

    /* Other */
    --border-radius: 8px;
    --max-width: 1200px;
    --shadow-sm: 0 2px 4px rgba(0,0,0,0.1);
    --shadow-md: 0 4px 12px rgba(0,0,0,0.15);
}
```

Using Variables

```css
header {
    background-color: var(--primary);
    padding: var(--space-md);
    color: var(--white);
}

.btn {
    background-color: var(--accent);
    padding: var(--space-sm) var(--space-md);
    border-radius: var(--border-radius);
}
```

Teaching Moment: Change ONE variable, the entire site updates. Want a different primary color? Change --primary at the top - every header, every button, every link uses the new color. This is the SECRET to maintainable CSS.

Step 4: Styling the Header & Navigation

```css
/* ============================
   HEADER
   ============================ */
header {
    background-color: var(--primary);
    color: var(--white);
    padding: var(--space-md) var(--space-lg);
    box-shadow: var(--shadow-sm);
    position: sticky;
    top: 0;
    z-index: 100;
}

header h1 {
    font-family: 'Playfair Display', Georgia, serif;
    font-size: 2rem;
    margin-bottom: var(--space-sm);
}

/* Navigation links */
nav a {
    color: var(--white);
    text-decoration: none;
    padding: var(--space-xs) var(--space-sm);
    margin-right: var(--space-sm);
    border-radius: 4px;
    transition: background-color 0.3s, color 0.3s;
```

```
}

nav a:hover {
    background-color: var(--accent);
    color: var(--primary);
}
```

Teaching New CSS Properties

position: sticky

Combined with top: 0 - the header STICKS to the top when user scrolls. Almost every modern website does this. Try Razorpay's homepage - notice the navbar follows you down the page.

z-index

Layering. Higher numbers go ON TOP. Header with z-index: 100 stays above other content when scrolling. Without it, content might cover the navbar.

text-decoration: none

Removes the underline from links. Default <a> tags are underlined - we don't always want that.

transition

Makes property changes SMOOTH instead of instant. transition: background-color 0.3s means 'when background-color changes, animate over 0.3 seconds'. Hover effects with transitions feel professional.

Step 5: The Hero Section

```
CSS
/* ============================
   HERO SECTION
   ============================ */
.hero {
    background: linear-gradient(135deg, #5B3A29 0%, #8B5E3C 100%);
    color: var(--white);
    text-align: center;
    padding: var(--space-xl) var(--space-md);
    min-height: 400px;
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
}

.hero h2 {
    font-family: 'Playfair Display', Georgia, serif;
    font-size: 3rem;
    margin-bottom: var(--space-sm);
}
```

```
.hero p {
    font-size: 1.25rem;
    margin-bottom: var(--space-lg);
    opacity: 0.9;
}

.hero .btn {
    background-color: var(--accent);
    color: var(--primary);
    padding: var(--space-sm) var(--space-lg);
    border-radius: var(--border-radius);
    text-decoration: none;
    font-weight: bold;
    transition: transform 0.2s, box-shadow 0.2s;
}

.hero .btn:hover {
    transform: translateY(-2px);
    box-shadow: var(--shadow-md);
}
```

linear-gradient()

Creates a smooth color transition. linear-gradient(135deg, color1, color2) means 'gradient at 135 degrees from color1 to color2'. Used for stunning backgrounds without images.

transform: translateY(-2px)

Moves element UP by 2 pixels. Combined with shadow on hover, creates a 'lifting' effect. Tiny detail, looks premium.

Step 6: Featured Books Section

```
CSS
/* ============================
   FEATURED BOOKS
   ============================ */
.featured {
    padding: var(--space-xl) var(--space-md);
    max-width: var(--max-width);
    margin: 0 auto;
}

.featured h2 {
    font-family: 'Playfair Display', Georgia, serif;
    font-size: 2.5rem;
    color: var(--primary);
```

```
    text-align: center;
    margin-bottom: var(--space-lg);
}

.book-card {
    background: var(--white);
    border-radius: var(--border-radius);
    padding: var(--space-md);
    box-shadow: var(--shadow-sm);
    text-align: center;
    transition: transform 0.3s, box-shadow 0.3s;
    margin-bottom: var(--space-md);
}

.book-card:hover {
    transform: translateY(-5px);
    box-shadow: var(--shadow-md);
}

.book-card img {
    width: 100%;
    height: 250px;
    object-fit: cover;
    border-radius: 4px;
    margin-bottom: var(--space-sm);
}

.book-card h3 {
    font-family: 'Playfair Display', Georgia, serif;
    color: var(--primary);
    margin-bottom: var(--space-xs);
}

.book-card .author {
    color: var(--text-muted);
    margin-bottom: var(--space-sm);
}

.book-card .price {
    font-size: 1.5rem;
    font-weight: bold;
    color: var(--accent);
    margin-bottom: var(--space-sm);
}

.book-card button {
    background-color: var(--primary);
    color: var(--white);
```

```
    border: none;
    padding: var(--space-sm) var(--space-md);
    border-radius: var(--border-radius);
    cursor: pointer;
    width: 100%;
    transition: background-color 0.3s;
}

.book-card button:hover {
    background-color: #8B5E3C;
}
```

max-width and margin: 0 auto

max-width: 1200px means 'don't grow wider than 1200px even on huge monitors'. margin: 0 auto centers the element horizontally. This combo creates the centered-content pattern used by every modern site.

object-fit: cover

For images. Makes the image FILL the container without distortion. Excess image is cropped. Without this, images stretch and look weird.

Step 7: Footer Styling

```
CSS
/* ============================
   FOOTER
   ============================ */
footer {
    background-color: var(--primary);
    color: var(--white);
    text-align: center;
    padding: var(--space-lg) var(--space-md);
    margin-top: var(--space-xl);
}

footer p {
    margin-bottom: var(--space-xs);
}

footer a {
    color: var(--accent);
    text-decoration: none;
    margin: 0 var(--space-xs);
}

footer a:hover {
    text-decoration: underline;
```

```
}
```

Topics Taught in This Phase

- Linking CSS to HTML with <link>
- Universal selector (*) and CSS reset
- box-sizing: border-box (CRITICAL)
- CSS Variables (--name) and var() function
- Element, class (.), and ID (#) selectors
- Pseudo-classes (:hover, :focus)
- Box model: margin, padding, border
- Typography: font-family, font-size, font-weight, line-height
- Colors: hex, rgb, transparency, linear-gradient
- Position: sticky, z-index
- Transitions and transforms for smooth interactions
- object-fit for responsive images
- max-width + margin: 0 auto for centered content

Today's Tasks for Students

- Add the complete style.css to your project.
- Link style.css in ALL 8 HTML pages.
- Change the CSS variable values to YOUR color scheme.
- Add hover effects to at least 3 different elements.
- Add a linear-gradient to one section.
- Commit: 'Phase 6 - Home page styled, brand identity established'.

PHASE 7: CSS FLEXBOX - NAVBAR & CARD LAYOUTS

Topic Introduction: One-Dimensional Layouts

Yesterday the home page started looking decent. But notice - the navigation links are stacked one on top of the other, not in a horizontal row. The book cards are stacked instead of in a grid. That's because we haven't told CSS how to ARRANGE them.

Today we learn Flexbox - the modern way to arrange elements in a row or column. By the end of this lesson, the home page looks PROFESSIONAL.

Step 1: Fix the Navbar with Flexbox

Update the header section in style.css:

```css
/* ============================
   HEADER WITH FLEXBOX
   ============================ */
header {
    background-color: var(--primary);
    color: var(--white);
    padding: var(--space-md) var(--space-lg);
    box-shadow: var(--shadow-sm);
    position: sticky;
    top: 0;
    z-index: 100;

    /* FLEXBOX MAGIC */
    display: flex;
    justify-content: space-between;
    align-items: center;
    flex-wrap: wrap;
    gap: var(--space-md);
}

header h1 {
    font-family: 'Playfair Display', Georgia, serif;
    font-size: 2rem;
    margin-bottom: 0;  /* removed - flexbox handles spacing */
}

/* Navigation as flex container too */
nav {
    display: flex;
    gap: var(--space-sm);
    flex-wrap: wrap;
}
```

```
nav a {
    color: var(--white);
    text-decoration: none;
    padding: var(--space-xs) var(--space-sm);
    border-radius: 4px;
    transition: all 0.3s;
}

nav a:hover {
    background-color: var(--accent);
    color: var(--primary);
}
```

Save. Refresh browser. Watch the navbar transform. The logo is now on the LEFT, the menu links are on the RIGHT, and everything is vertically centered.

Teaching Flexbox - The 3 Magic Properties

display: flex

This is the SWITCH. Adding 'display: flex' to a parent activates Flexbox. All direct children become 'flex items' that can be arranged.

justify-content

Controls HORIZONTAL spacing (in a row):

- flex-start: items grouped at the left (default)
- center: items grouped in the center
- flex-end: items grouped at the right
- space-between: first item left, last item right, equal space between
- space-around: equal space around each item
- space-evenly: equal space everywhere

align-items

Controls VERTICAL alignment:

- center: vertically centered (most common)
- flex-start: aligned to top
- flex-end: aligned to bottom
- stretch: items stretch to fill height

gap

Adds space BETWEEN items. Like margin but cleaner. gap: 20px = 20px of space between all flex items.

□ Teaching Moment: These three properties solve 90% of layout problems. justify-content for horizontal, align-items for vertical, gap for spacing. Memorize this trio.

Step 2: Featured Books as a Flex Grid

Update the featured section CSS:

```css
/* ============================
   FEATURED BOOKS GRID
   ============================ */
.featured {
    padding: var(--space-xl) var(--space-md);
    max-width: var(--max-width);
    margin: 0 auto;
}

.featured h2 {
    font-family: 'Playfair Display', Georgia, serif;
    font-size: 2.5rem;
    color: var(--primary);
    text-align: center;
    margin-bottom: var(--space-lg);
}

/* MAKE FEATURED INTO A FLEXBOX CONTAINER */
.featured {
    display: flex;
    flex-wrap: wrap;
    justify-content: center;
    gap: var(--space-lg);
}

/* But wait - we need the heading on its own row */
/* Solution: wrap the h2 separately, OR use a wrapper for cards */
```

Hmm, we hit a problem. The h2 heading becomes a flex item too. Let's fix the HTML structure first:

Improve the HTML Structure

```html
<!-- Update in index.html -->
<section class="featured">
    <h2>Featured Books</h2>

    <!-- Wrapper div for the cards -->
    <div class="book-grid">

        <article class="book-card">
            <img src="images/books/wings-of-fire.jpg" alt="Wings of Fire">
            <h3>Wings of Fire</h3>
            <p class="author">A.P.J. Abdul Kalam</p>
```

```
            <p class="price">₹299</p>
            <button>Add to Cart</button>
        </article>

        <!-- More book cards... -->

    </div>
</section>
```

Now the CSS is cleaner:

```
CSS
.book-grid {
    display: flex;
    flex-wrap: wrap;
    justify-content: center;
    gap: var(--space-lg);
    margin-top: var(--space-lg);
}

.book-card {
    background: var(--white);
    border-radius: var(--border-radius);
    padding: var(--space-md);
    box-shadow: var(--shadow-sm);
    text-align: center;
    transition: transform 0.3s, box-shadow 0.3s;

    /* FLEX ITEM CONFIG */
    flex: 1 1 250px;  /* grow, shrink, base width */
    max-width: 280px;
}

.book-card:hover {
    transform: translateY(-5px);
    box-shadow: var(--shadow-md);
}
```

Understanding flex: 1 1 250px

This is shorthand for THREE properties:

- **flex-grow: 1:** Can grow if extra space is available.
- **flex-shrink: 1:** Can shrink if space is tight.
- **flex-basis: 250px:** Start at 250px wide.

Result: cards naturally fit the container, grow/shrink as needed, and never go below 250px.

Step 3: Categories Section with Flexbox

```css
/* ============================
   CATEGORIES
   ============================ */
.categories {
    background-color: var(--light-gray);
    padding: var(--space-xl) var(--space-md);
}

.categories h2 {
    font-family: 'Playfair Display', Georgia, serif;
    font-size: 2.5rem;
    color: var(--primary);
    text-align: center;
    margin-bottom: var(--space-lg);
    max-width: var(--max-width);
    margin-left: auto;
    margin-right: auto;
}

.category-list-container {
    display: flex;
    flex-wrap: wrap;
    justify-content: center;
    gap: var(--space-md);
    max-width: var(--max-width);
    margin: 0 auto;
}

.category {
    background: var(--white);
    padding: var(--space-lg);
    border-radius: var(--border-radius);
    text-align: center;
    box-shadow: var(--shadow-sm);
    flex: 1 1 200px;
    max-width: 240px;
    cursor: pointer;
    transition: all 0.3s;
}

.category:hover {
    background-color: var(--accent);
    color: var(--primary);
    transform: scale(1.05);
}

.category h3 {
```

```
    font-family: 'Playfair Display', Georgia, serif;
    margin-bottom: var(--space-sm);
}
```

Don't forget to wrap the category divs in the HTML:

```HTML
<section class="categories">
    <h2>Browse by Category</h2>

    <div class="category-list-container">
        <div class="category">
            <h3>📖 Fiction</h3>
            <p>Novels, short stories, fantasy</p>
        </div>
        <!-- More categories... -->
    </div>
</section>
```

Step 4: Footer with Flexbox

```CSS
/* ============================
   FOOTER WITH COLUMNS
   ============================ */
footer {
    background-color: var(--primary);
    color: var(--white);
    padding: var(--space-xl) var(--space-md);
    margin-top: var(--space-xl);
}

.footer-content {
    display: flex;
    flex-wrap: wrap;
    gap: var(--space-lg);
    max-width: var(--max-width);
    margin: 0 auto;
}

.footer-column {
    flex: 1 1 200px;
}

.footer-column h4 {
    font-family: 'Playfair Display', Georgia, serif;
    margin-bottom: var(--space-md);
```

```
    color: var(--accent);
}

.footer-column ul {
    list-style: none;
}

.footer-column li {
    margin-bottom: var(--space-xs);
}

.footer-column a {
    color: var(--white);
    text-decoration: none;
    transition: color 0.3s;
}

.footer-column a:hover {
    color: var(--accent);
}

.footer-bottom {
    text-align: center;
    padding-top: var(--space-lg);
    margin-top: var(--space-lg);
    border-top: 1px solid rgba(255,255,255,0.2);
}
```

Update the footer HTML to have proper columns:

HTML

```
<footer>
    <div class="footer-content">

        <div class="footer-column">
            <h4>📖 BookHive</h4>
            <p>India's favorite online bookstore.</p>
        </div>

        <div class="footer-column">
            <h4>Quick Links</h4>
            <ul>
                <li><a href="index.html">Home</a></li>
                <li><a href="books.html">Books</a></li>
                <li><a href="about.html">About</a></li>
                <li><a href="contact.html">Contact</a></li>
            </ul>
        </div>
```

```
        <div class="footer-column">
            <h4>Categories</h4>
            <ul>
                <li><a href="#">Fiction</a></li>
                <li><a href="#">Non-Fiction</a></li>
                <li><a href="#">Biographies</a></li>
                <li><a href="#">Business</a></li>
            </ul>
        </div>

        <div class="footer-column">
            <h4>Follow Us</h4>
            <ul>
                <li><a href="#">  Facebook</a></li>
                <li><a href="#">  Instagram</a></li>
                <li><a href="#">  Twitter</a></li>
                <li><a href="#">  LinkedIn</a></li>
            </ul>
        </div>

    </div>

    <div class="footer-bottom">
        <p>&copy; 2026 BookHive. All rights reserved.</p>
    </div>
</footer>
```

Topics Taught in This Phase

- display: flex - activating Flexbox
- justify-content - main axis alignment
- align-items - cross axis alignment
- flex-wrap - allowing items to wrap to next line
- gap - spacing between flex items
- flex shorthand (flex-grow, flex-shrink, flex-basis)
- Nested flex containers (flex inside flex)
- Real-world layouts: navbar, card grid, multi-column footer

Today's Tasks for Students

- Convert the navbar to use Flexbox.
- Convert featured books to a flex grid.
- Convert categories to a flex grid.

- Build a 4-column footer with Flexbox.
- Play Flexbox Froggy (flexboxfroggy.com) - all 24 levels.
- Commit: 'Phase 7 - Flexbox layouts applied to entire home page'.

PHASE 8: CSS GRID & RESPONSIVE - BOOKS LISTING PAGE

Topic Introduction: Grid + Responsiveness

Flexbox is great for one-dimensional layouts. But for a true GRID like the books listing page - rows AND columns - CSS Grid is the king. Plus, we make EVERYTHING responsive today. By the end, the site looks great on phones, tablets, and laptops.

Step 1: CSS Grid for Books Listing

Open books.html. Add the link to a new CSS file:

```HTML
<link rel="stylesheet" href="css/style.css">
<link rel="stylesheet" href="css/books.css">
```

Now create css/books.css with the grid styles:

```CSS
/* ============================
   BOOKS PAGE STYLES
   ============================ */

.page-header {
    background: linear-gradient(135deg, #5B3A29 0%, #8B5E3C 100%);
    color: var(--white);
    text-align: center;
    padding: var(--space-xl) var(--space-md);
}

.page-header h2 {
    font-family: 'Playfair Display', Georgia, serif;
    font-size: 2.5rem;
    margin-bottom: var(--space-sm);
}

/* ============================
   SEARCH & FILTERS
   ============================ */
.filters {
    background: var(--white);
    padding: var(--space-md);
    box-shadow: var(--shadow-sm);
    margin-bottom: var(--space-lg);
}

.search-box {
```

```
    display: flex;
    gap: var(--space-sm);
    max-width: 600px;
    margin: 0 auto var(--space-md);
}

.search-box input {
    flex: 1;
    padding: var(--space-sm);
    border: 2px solid #ddd;
    border-radius: var(--border-radius);
    font-size: 1rem;
}

.search-box input:focus {
    outline: none;
    border-color: var(--primary);
}

.search-box button {
    background: var(--primary);
    color: var(--white);
    border: none;
    padding: var(--space-sm) var(--space-lg);
    border-radius: var(--border-radius);
    cursor: pointer;
}

.filter-controls {
    display: flex;
    gap: var(--space-md);
    justify-content: center;
    flex-wrap: wrap;
}

.filter-controls select {
    padding: var(--space-sm);
    border: 1px solid #ddd;
    border-radius: var(--border-radius);
}

/* ============================
   THE GRID - The Star of This Phase
   ============================ */
.books-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
    gap: var(--space-lg);
```

```
    padding: var(--space-md);
    max-width: var(--max-width);
    margin: 0 auto;
}

.book-card {
    background: var(--white);
    border-radius: var(--border-radius);
    padding: var(--space-md);
    box-shadow: var(--shadow-sm);
    text-align: center;
    transition: all 0.3s;
}

.book-card:hover {
    transform: translateY(-5px);
    box-shadow: var(--shadow-md);
}

.book-card img {
    width: 100%;
    height: 220px;
    object-fit: cover;
    border-radius: 4px;
    margin-bottom: var(--space-sm);
}

.book-card h3 {
    font-family: 'Playfair Display', Georgia, serif;
    color: var(--primary);
    font-size: 1.1rem;
    margin-bottom: var(--space-xs);
}

.book-card .author {
    color: var(--text-muted);
    font-size: 0.9rem;
    margin-bottom: var(--space-xs);
}

.book-card .rating {
    color: var(--accent);
    font-size: 0.9rem;
    margin-bottom: var(--space-sm);
}

.book-card .price {
    font-size: 1.4rem;
```

```
    font-weight: bold;
    color: var(--primary);
    margin-bottom: var(--space-sm);
}

.book-card button {
    width: 100%;
    background: var(--primary);
    color: var(--white);
    border: none;
    padding: var(--space-sm);
    border-radius: var(--border-radius);
    cursor: pointer;
    transition: background 0.3s;
}

.book-card button:hover {
    background: var(--accent);
    color: var(--primary);
}
```

The Magic Line - Memorize It

```
CSS
grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
```

This ONE line creates a fully responsive grid. Breaking it down:

- **auto-fit:** Fit as many columns as possible in the available space.
- **minmax(220px, 1fr):** Each column is AT LEAST 220px, but can grow to 1 fraction of space.

Result on different screens:

- Mobile (320px): 1 column (220px wide)
- Tablet (768px): 3 columns
- Laptop (1200px): 5 columns
- All without media queries!

? Teaching Moment: This single line replaces 50+ lines of old-school responsive CSS. Modern CSS Grid is INCREDIBLE. Every senior developer knows this pattern. Now your students do too.

Step 2: Responsive Design with Media Queries

Even though Grid is responsive, we need media queries for OTHER elements - text size, padding, hiding things on mobile. Create css/responsive.css:

```
CSS
/* ============================
   RESPONSIVE DESIGN
```

```
   ============================ */

/* ============================
   TABLET (768px and below)
   ============================ */
@media (max-width: 768px) {

    /* Header - stack on mobile */
    header {
        flex-direction: column;
        text-align: center;
        gap: var(--space-sm);
    }

    header h1 {
        font-size: 1.5rem;
    }

    nav {
        justify-content: center;
        gap: var(--space-xs);
    }

    nav a {
        font-size: 0.9rem;
        padding: 4px 8px;
    }

    /* Hero - smaller text */
    .hero h2 {
        font-size: 2rem;
    }

    .hero p {
        font-size: 1rem;
    }

    .hero {
        padding: var(--space-lg) var(--space-md);
        min-height: 300px;
    }

    /* Featured books - tighter grid */
    .featured h2 {
        font-size: 2rem;
    }
```

```
    /* Filters - stack on mobile */
    .filter-controls {
        flex-direction: column;
    }

    /* Cart table - hide some columns on mobile */
    .cart-table thead {
        display: none;
    }

    .cart-table tbody tr {
        display: block;
        margin-bottom: var(--space-md);
        background: var(--white);
        padding: var(--space-md);
        border-radius: var(--border-radius);
    }

    .cart-table td {
        display: block;
        text-align: left;
        padding: var(--space-xs) 0;
    }
}

/* ============================
   MOBILE (480px and below)
   ============================ */
@media (max-width: 480px) {

    /* Even tighter on phones */
    header h1 {
        font-size: 1.25rem;
    }

    .hero h2 {
        font-size: 1.5rem;
    }

    .book-card img {
        height: 180px;
    }

    .footer-content {
        flex-direction: column;
        text-align: center;
    }
```

```
}

/* ============================
   LARGE DESKTOPS (1400px+)
   ============================ */
@media (min-width: 1400px) {

    .hero h2 {
        font-size: 4rem;
    }

    .featured h2,
    .categories h2 {
        font-size: 3rem;
    }
}
```

Link responsive.css in all your HTML pages (after style.css):

```HTML
<link rel="stylesheet" href="css/style.css">
<link rel="stylesheet" href="css/responsive.css">
```

Teaching Media Queries

The Mobile-First vs Desktop-First Choice

- **Mobile-first:** Default styles work on mobile. Use min-width to add styles for larger screens.
- **Desktop-first:** Default styles work on desktop. Use max-width to scale down for smaller screens.

We're using desktop-first here (max-width). Industry slightly prefers mobile-first, but both work.

Common Breakpoints

- 320px - small phones
- 480px - large phones
- 768px - tablets
- 1024px - small laptops
- 1200px+ - desktops

Step 3: Test Responsiveness

In Chrome, open DevTools (F12). Click the 'device toolbar' icon (looks like a phone/tablet). Select different devices:

- iPhone SE (375px)
- iPhone 12 Pro (390px)
- iPad (768px)
- iPad Pro (1024px)

Common Issue

If a phone shows your site weirdly small (zoomed out), check the viewport meta tag in HTML. <meta name="viewport" content="width=device-width, initial-scale=1.0"> is REQUIRED for responsive sites.

Step 4: Add Animations

Bring the page alive with subtle animations. Add to style.css:

```css
/* ============================
   ANIMATIONS
   ============================ */

/* Fade in on page load */
@keyframes fadeIn {
    from {
        opacity: 0;
        transform: translateY(20px);
    }
    to {
        opacity: 1;
        transform: translateY(0);
    }
}

.hero h2,
.hero p,
.hero .btn {
    animation: fadeIn 0.8s ease-out forwards;
}

.hero p {
    animation-delay: 0.2s;
    opacity: 0;
}

.hero .btn {
    animation-delay: 0.4s;
    opacity: 0;
}

/* Pulse for sale tags */
@keyframes pulse {
    0%, 100% { transform: scale(1); }
    50% { transform: scale(1.05); }
```

```
}

.sale-tag {
    background: var(--error);
    color: var(--white);
    padding: 4px 8px;
    border-radius: 4px;
    font-size: 0.8rem;
    display: inline-block;
    animation: pulse 2s infinite;
}

/* Loading spinner */
@keyframes spin {
    from { transform: rotate(0deg); }
    to { transform: rotate(360deg); }
}

.spinner {
    width: 40px;
    height: 40px;
    border: 4px solid #f3f3f3;
    border-top-color: var(--primary);
    border-radius: 50%;
    animation: spin 1s linear infinite;
    margin: var(--space-lg) auto;
}
```

Teaching @keyframes

Defines an animation. 'from' is the start state, 'to' is the end. You can also use 0%, 50%, 100% for multiple steps. The animation property applies it to elements.

Topics Taught in This Phase

- CSS Grid - display: grid
- grid-template-columns with repeat(), auto-fit, minmax()
- The magic responsive grid formula
- Media queries - @media (max-width: 768px)
- Mobile-first vs Desktop-first approaches
- Common breakpoints (320, 480, 768, 1024, 1200)
- @keyframes for animations
- animation property with duration, easing, delay, iteration
- Pulse, spin, and fade-in animations
- Testing in Chrome DevTools device mode

Today's Tasks for Students

- Apply CSS Grid to the books listing page.
- Add responsive.css with breakpoints for tablet and mobile.
- Test the site on at least 3 different screen sizes.
- Add at least 2 keyframe animations.
- Play CSS Grid Garden (cssgridgarden.com) - all 28 levels.
- Commit: 'Phase 8 - Responsive grid layout + animations'.

> **CSS Phase Complete**
>
> Your site now looks BEAUTIFUL and works on every device. The visual design is done. From Phase 9 onwards, we add INTERACTIVITY - the JavaScript that makes it ALIVE.

PHASE 9: JAVASCRIPT BASICS - FIRST INTERACTIONS

Topic Introduction: Bringing the Site to Life

The site LOOKS amazing but DOES nothing yet. Click 'Add to Cart' - nothing. Click the cart - just empty page. Today, JavaScript starts changing that.

Today's focus: JavaScript fundamentals + connecting JS to HTML. We'll start with simple interactions.

Step 1: Link JavaScript to HTML

Open index.html. Add this RIGHT BEFORE the closing </body> tag:

```HTML
<script src="js/main.js"></script>
```

Why at the End of <body>?

JavaScript loads top-to-bottom. If we put it in <head>, the script tries to find HTML elements that don't exist yet. Putting it at the END means all HTML loads first, then JS runs - and can find every element.

Step 2: Your First JS Code

Open js/main.js. Add:

```JAVASCRIPT
// Welcome message
console.log("Welcome to BookHive! ");

// Show alert when page loads
// alert("BookHive is loading!");
```

Refresh the browser. Press F12. Click 'Console' tab. You'll see 'Welcome to BookHive! '. Uncomment the alert line to see a popup.

Teaching Moment: This is huge. Students are now SEEING their JavaScript run. They wrote code, it executed in a browser. This is real development.

Step 3: Variables - Storing BookHive Data

```JAVASCRIPT
// ====================
// VARIABLES - Store Data
// ====================

// const - for values that won't change
```

```
const SITE_NAME = "BookHive";
const FOUNDED_YEAR = 2026;
const PI = 3.14159;

// let - for values that WILL change
let cartCount = 0;
let totalAmount = 0;
let currentUser = null;

// Print values
console.log(SITE_NAME);
console.log("Founded:", FOUNDED_YEAR);
console.log("Cart count:", cartCount);

// Change let variables
cartCount = 3;
totalAmount = 1497;
console.log("New cart count:", cartCount);
console.log("Total amount: ₹" + totalAmount);

// Try changing a const - will ERROR
// SITE_NAME = "BookStore";  // Error!
```

Teaching const vs let

- **const:** Use by default. For values that don't change after declaration.
- **let:** Only when value WILL change (counters, user inputs, etc.).
- **var:** OLD way. Avoid. You'll see it in old code only.

Step 4: Data Types

```
JAVASCRIPT
// ====================
// DATA TYPES IN JAVASCRIPT
// ====================

// STRING - text
const bookTitle = "Wings of Fire";
const author = 'A.P.J. Abdul Kalam';        // single quotes work too
const description = `Bestseller since 1999`;  // backticks for template literals

// NUMBER - any number
const price = 299;
const discount = 0.20;        // 20%
const rating = 4.8;
```

```
// BOOLEAN - true or false
const isInStock = true;
const isOnSale = false;

// NULL - intentionally empty
const couponCode = null;

// UNDEFINED - not yet set
let promoMessage;             // undefined

// Check the type
console.log(typeof bookTitle);  // "string"
console.log(typeof price);       // "number"
console.log(typeof isInStock);   // "boolean"
console.log(typeof promoMessage); // "undefined"
```

Step 5: Template Literals - Smart Strings

```
JAVASCRIPT
// OLD way - hard to read
const message1 = "Hello, " + author + "! Your book '" + bookTitle + "' costs ₹" +
price;

// NEW way with template literals (backticks + ${})
const message2 = `Hello, ${author}! Your book '${bookTitle}' costs ₹${price}`;

console.log(message1);
console.log(message2);

// Multi-line strings work too
const productCard = `
    <div class="book-card">
        <h3>${bookTitle}</h3>
        <p>by ${author}</p>
        <p>₹${price}</p>
    </div>
`;
console.log(productCard);
```

Teaching Moment: Template literals are CRITICAL for the upcoming phases. When we generate book cards dynamically, this is how we'll write the HTML.

Step 6: Math Operations

```
JAVASCRIPT
```

```
// ====================
// MATH FOR BOOKHIVE
// ====================

const bookPrice = 299;
const quantity = 3;

// Basic math
const subtotal = bookPrice * quantity;           // 897
const tax = subtotal * 0.05;                     // 5% GST
const total = subtotal + tax;                    // 941.85
const discount = total * 0.10;                   // 10% off
const finalPrice = total - discount;             // 847.665

console.log("Subtotal:", subtotal);
console.log("Tax:", tax);
console.log("Total:", total);
console.log("After discount:", finalPrice);

// Round to 2 decimals (for currency)
console.log("Final:", finalPrice.toFixed(2));    // "847.67"

// Math object functions
console.log(Math.round(4.7));    // 5 (rounds to nearest)
console.log(Math.ceil(4.2));     // 5 (always rounds up)
console.log(Math.floor(4.9));    // 4 (always rounds down)
console.log(Math.max(10, 20));   // 20
console.log(Math.min(10, 20));   // 10
console.log(Math.random());       // random 0-1
```

Step 7: First Real Interaction - Changing the Page

Now let's USE JavaScript to actually change the page. Let's update the welcome message dynamically:

HTML - Add ID to Element

```
HTML
<!-- In index.html, update the hero section -->
<section class="hero">
    <h2 id="hero-title">Welcome to BookHive</h2>
    <p id="hero-subtitle">India's favorite place to discover great books</p>
    <a href="books.html" class="btn">Browse Books</a>
</section>
```

JavaScript - Change the Text

```
JAVASCRIPT
```

```
// ====================
// DOM MANIPULATION - First Time!
// ====================

// 1. Find the element on the page using its ID
const heroTitle = document.getElementById("hero-title");
const heroSubtitle = document.getElementById("hero-subtitle");

// 2. Read its current text
console.log("Current title:", heroTitle.textContent);

// 3. CHANGE the text
heroTitle.textContent = "Welcome to BookHive";

// Get current time and personalize
const currentHour = new Date().getHours();
let greeting;

if (currentHour < 12) {
    greeting = "Good Morning! ";
} else if (currentHour < 17) {
    greeting = "Good Afternoon! ";
} else {
    greeting = "Good Evening! ";
}

heroSubtitle.textContent = `${greeting} Find your next great read.`;
```

Save and refresh. The greeting changes based on the time of day! This is the BEGINNING of interactive websites.

Step 8: Selecting Elements - The 3 Main Ways

```
JAVASCRIPT
// ====================
// 3 WAYS TO SELECT ELEMENTS
// ====================

// 1. By ID (unique)
const heroTitle = document.getElementById("hero-title");

// 2. By class (multiple)
const allBookCards = document.getElementsByClassName("book-card");

// 3. Modern way - querySelector (uses CSS selectors!)
const firstCard = document.querySelector(".book-card");
const titleByQuery = document.querySelector("#hero-title");
const allCardsQuery = document.querySelectorAll(".book-card");
```

```
// querySelector returns FIRST match
// querySelectorAll returns ALL matches

console.log("Number of book cards:", allCardsQuery.length);
```

Best Practice

Use querySelector and querySelectorAll. They use CSS-style selectors (.class, #id, [attr]) - familiar syntax. Modern code rarely uses getElementById.

Step 9: Update the Cart Count

Show students how to update the cart count in the navbar:

HTML - Add ID to Cart Link

```html
<!-- In all HTML pages, update the cart link in nav -->
<a href="cart.html" id="cart-link">🛒 Cart (<span id="cart-count">0</span>)</a>
```

JavaScript - Update Count

```javascript
// js/main.js

let cartCount = 0;

function updateCartCount() {
    const cartCountElement = document.getElementById("cart-count");
    if (cartCountElement) {
        cartCountElement.textContent = cartCount;
    }
}

// Test it - let's pretend the cart has 3 items
cartCount = 3;
updateCartCount();

// The navbar now shows "🛒 Cart (3)"
```

Topics Taught in This Phase

- Adding JavaScript to HTML with <script src>
- Why scripts go at end of <body>
- console.log() and the browser console

- Variables: const, let, var
- Data types: string, number, boolean, null, undefined
- Template literals with backticks and ${}
- Math operations and Math object
- toFixed() for currency formatting
- DOM selection: getElementById, querySelector, querySelectorAll
- textContent for changing element text
- Date object for time-based logic

Today's Tasks for Students

- Link main.js to all 8 HTML pages.
- Practice all the variable and data type examples.
- Use template literals to build at least 3 dynamic strings.
- Try all 3 element selection methods on different elements.
- Add a time-based greeting to your hero section.
- Commit: 'Phase 9 - JavaScript basics, first DOM interactions'.

PHASE 10: JS ARRAYS & OBJECTS - DYNAMIC BOOK DATA

Topic Introduction: Real Data, Real Pages

Right now, book cards are hard-coded in HTML. Every new book = manually write HTML. Bad approach. Today: we store all books in JavaScript and GENERATE the HTML automatically.

This is exactly how every real website works. Swiggy doesn't hard-code restaurants in HTML. They have a database of restaurants and JavaScript generates the cards. That's what we're learning.

Step 1: Create the Book Database

Open js/books.js. This is our 'database' - a JavaScript array of book objects:

```javascript
// ====================
// BOOK DATABASE
// js/books.js
// ====================

const books = [
    {
        id: 1,
        title: "Wings of Fire",
        author: "A.P.J. Abdul Kalam",
        category: "biography",
        price: 299,
        originalPrice: 499,
        rating: 4.8,
        reviews: 2341,
        image: "images/books/wings-of-fire.jpg",
        description: "Autobiography of India's beloved former president.",
        inStock: true
    },
    {
        id: 2,
        title: "The Alchemist",
        author: "Paulo Coelho",
        category: "fiction",
        price: 349,
        originalPrice: 499,
        rating: 4.7,
        reviews: 5012,
        image: "images/books/the-alchemist.jpg",
        description: "A magical story about pursuing your dreams.",
        inStock: true
    },
    {
```

```
        id: 3,
        title: "Atomic Habits",
        author: "James Clear",
        category: "self-help",
        price: 499,
        originalPrice: 799,
        rating: 4.9,
        reviews: 8420,
        image: "images/books/atomic-habits.jpg",
        description: "Tiny changes, remarkable results.",
        inStock: true
    },
    {
        id: 4,
        title: "Sapiens",
        author: "Yuval Noah Harari",
        category: "history",
        price: 599,
        originalPrice: 899,
        rating: 4.6,
        reviews: 12450,
        image: "images/books/sapiens.jpg",
        description: "A brief history of humankind.",
        inStock: true
    },
    {
        id: 5,
        title: "Rich Dad Poor Dad",
        author: "Robert Kiyosaki",
        category: "business",
        price: 350,
        originalPrice: 499,
        rating: 4.5,
        reviews: 6800,
        image: "images/books/rich-dad.jpg",
        description: "What the rich teach their kids about money.",
        inStock: true
    },
    {
        id: 6,
        title: "Five Point Someone",
        author: "Chetan Bhagat",
        category: "fiction",
        price: 199,
        originalPrice: 299,
        rating: 4.3,
        reviews: 9200,
        image: "images/books/five-point.jpg",
        description: "What not to do at IIT.",
```

```
        inStock: false
    },
    {
        id: 7,
        title: "The Power of Subconscious Mind",
        author: "Joseph Murphy",
        category: "self-help",
        price: 249,
        originalPrice: 399,
        rating: 4.6,
        reviews: 4100,
        image: "images/books/subconscious.jpg",
        description: "Unlock the power within you.",
        inStock: true
    },
    {
        id: 8,
        title: "Think and Grow Rich",
        author: "Napoleon Hill",
        category: "business",
        price: 299,
        originalPrice: 499,
        rating: 4.7,
        reviews: 7300,
        image: "images/books/think-grow.jpg",
        description: "The classic guide to success.",
        inStock: true
    }
];

console.log(`Loaded ${books.length} books`);
```

Teaching Objects and Arrays

Objects - Real-World Things

Each book is an OBJECT. Curly braces wrap properties. Each property is key: value. Real things in code = objects. A user is an object, a product is an object, an order is an object.

Arrays - Lists of Things

Square brackets wrap a LIST. Our 'books' is an array of book objects. Arrays have:

- books.length - how many items (8)
- books[0] - first book (Wings of Fire) - INDEX STARTS AT 0!
- books[7] - eighth book (Think and Grow Rich)
- Each item has properties accessed via dot: books[0].title

Step 2: Generate Book Cards from Data

This is where it gets exciting. Open js/main.js (or create books-page.js linked from books.html). Add:

JAVASCRIPT

```
// ====================
// GENERATE BOOK CARDS DYNAMICALLY
// ====================

// First, link books.js BEFORE this file in HTML
// <script src="js/books.js"></script>
// <script src="js/main.js"></script>

// Find the container where book cards should go
const booksContainer = document.getElementById("books-container");

// Function to create HTML for one book
function createBookCard(book) {
    return `
        <article class="book-card" data-id="${book.id}">
            <img src="${book.image}" alt="${book.title}" loading="lazy">
            <h3>${book.title}</h3>
            <p class="author">${book.author}</p>
            <p class="rating">⭐ ${book.rating} (${book.reviews} reviews)</p>
            <p class="price">
                ₹${book.price}
                <small><s>₹${book.originalPrice}</s></small>
            </p>
            <button
                class="add-to-cart"
                data-id="${book.id}"
                ${!book.inStock ? "disabled" : ""}
            >
                ${book.inStock ? "Add to Cart" : "Out of Stock"}
            </button>
        </article>
    `;
}

// Function to render all books to the page
function renderBooks(bookList) {
    if (!booksContainer) return;

    // map() each book to its HTML, then join all into one string
    const allHTML = bookList.map(book => createBookCard(book)).join("");

    // Replace the container's content
    booksContainer.innerHTML = allHTML;
}
```

```
// Render all books when page loads
renderBooks(books);
```

Update books.html - replace the hard-coded book cards with just an empty container:

```
HTML
<!-- In books.html -->
<section class="books-grid" id="books-container">
    <!-- Books will be generated by JavaScript -->
</section>

<!-- Link the scripts at the end -->
<script src="js/books.js"></script>
<script src="js/main.js"></script>
```

Save. Refresh. ALL 8 books appear automatically. Add a 9th book to books.js - it appears too. Magic.

Teaching Moment: This is the most important moment in the course. Students see how DATA + TEMPLATE = DYNAMIC HTML. This is React's core idea, served plain. Once they get this, React makes sense later.

Step 3: Array Methods - The BIG THREE

map() - Transform Each Item

```
JAVASCRIPT
// Get just the book titles
const titles = books.map(book => book.title);
console.log(titles);
// ["Wings of Fire", "The Alchemist", "Atomic Habits", ...]

// Get titles with prices
const titlesWithPrices = books.map(book =>
    `${book.title} - ₹${book.price}`
);
console.log(titlesWithPrices);
// ["Wings of Fire - ₹299", "The Alchemist - ₹349", ...]

// Calculate discount percentage for each book
const booksWithDiscount = books.map(book => ({
    ...book,
    discount: Math.round(((book.originalPrice - book.price) / book.originalPrice)
* 100)
}));
// Each book now has a 'discount' property like 40 (for 40% off)
```

filter() - Keep Items That Match

```
JAVASCRIPT
```

```
// Get only books in stock
const availableBooks = books.filter(book => book.inStock);
console.log(`${availableBooks.length} books in stock`);

// Get only fiction books
const fictionBooks = books.filter(book => book.category === "fiction");

// Get only books under ₹300
const cheapBooks = books.filter(book => book.price < 300);

// Get only highly rated books
const topRated = books.filter(book => book.rating >= 4.7);

// Search by name (case-insensitive)
const searchTerm = "atom";
const matching = books.filter(book =>
    book.title.toLowerCase().includes(searchTerm.toLowerCase())
);
console.log("Search results:", matching);
// [{ Atomic Habits ... }]
```

reduce() - Combine All Items into ONE

```
JAVASCRIPT
// Total price of all books
const totalPrice = books.reduce((sum, book) => sum + book.price, 0);
console.log("Total catalog value: ₹" + totalPrice);

// Average rating
const totalRating = books.reduce((sum, book) => sum + book.rating, 0);
const averageRating = totalRating / books.length;
console.log("Average rating:", averageRating.toFixed(1));

// Group by category
const byCategory = books.reduce((groups, book) => {
    if (!groups[book.category]) {
        groups[book.category] = [];
    }
    groups[book.category].push(book);
    return groups;
}, {});
console.log(byCategory);
// { biography: [...], fiction: [...], self-help: [...] }
```

Step 4: Sorting Books

```javascript
// Sort by price - low to high
const sortedByPriceLow = [...books].sort((a, b) => a.price - b.price);

// Sort by price - high to low
const sortedByPriceHigh = [...books].sort((a, b) => b.price - a.price);

// Sort by rating (highest first)
const sortedByRating = [...books].sort((a, b) => b.rating - a.rating);

// Sort by title alphabetically
const sortedByTitle = [...books].sort((a, b) =>
    a.title.localeCompare(b.title)
);

// IMPORTANT: [...books] creates a COPY before sorting
// sort() mutates the original array - we don't want that
```

The Spread Operator [...]

The three dots [...books] copies the array. Always copy before sorting to keep the original unchanged. This is called 'immutability' - never mutate, always create new.

Step 5: Find a Specific Book

```javascript
// find() returns the FIRST matching book
const wingsOfFire = books.find(book => book.id === 1);
console.log(wingsOfFire);

// findIndex() returns the position (0-based)
const index = books.findIndex(book => book.id === 5);
console.log("Found at index:", index);

// Check if a book exists
const hasAtomicHabits = books.some(book => book.title === "Atomic Habits");
console.log(hasAtomicHabits);  // true

// Check if ALL books have a rating above 4
const allWellRated = books.every(book => book.rating > 4);
console.log(allWellRated);  // true
```

Topics Taught in This Phase

- Objects - { key: value } structure
- Arrays - lists of items, zero-indexed

- Array of objects - the foundation of all data-driven apps
- Dynamic HTML generation with template literals
- map() - transform each item, return new array
- filter() - keep matching items
- reduce() - combine into single value
- sort() with compare function
- find(), findIndex(), some(), every()
- Spread operator [...] for copying
- innerHTML for replacing entire content

Today's Tasks for Students

- Create books.js with at least 12 books.
- Make sure books.html generates all cards from the array.
- Practice map, filter, reduce in the console.
- Write functions to: find cheapest book, find books in a category, count books in each category.
- Sort books by different criteria and log results.
- Commit: 'Phase 10 - Dynamic book generation from JavaScript data'.

PHASE 11: JS EVENTS - CART, SEARCH & INTERACTIVITY

Topic Introduction: Click, Type, Interact

Today the website becomes a real application. Clicking 'Add to Cart' WORKS. Typing in search FILTERS results. Selecting a category SHOWS only those books. All this through JavaScript Events.

Step 1: Click Events - Add to Cart

```javascript
// js/cart.js

// ====================
// CART STATE
// ====================
let cart = [];  // Empty cart to start

// Load cart from localStorage if exists
const savedCart = localStorage.getItem("bookhive-cart");
if (savedCart) {
    cart = JSON.parse(savedCart);
}

// ====================
// ADD TO CART
// ====================
function addToCart(bookId) {
    // Find the book in our database
    const book = books.find(b => b.id === bookId);

    if (!book) {
        alert("Book not found!");
        return;
    }

    if (!book.inStock) {
        alert("Sorry, this book is out of stock!");
        return;
    }

    // Check if already in cart
    const existingItem = cart.find(item => item.id === bookId);

    if (existingItem) {
        // Increase quantity
        existingItem.quantity += 1;
```

```
    } else {
        // Add new item
        cart.push({
            id: book.id,
            title: book.title,
            author: book.author,
            price: book.price,
            image: book.image,
            quantity: 1
        });
    }

    // Save to localStorage
    saveCart();

    // Update the cart count in navbar
    updateCartCount();

    // Show feedback to user
    showNotification(`Added "${book.title}" to cart! `);
}

// ====================
// SAVE CART TO LOCALSTORAGE
// ====================
function saveCart() {
    localStorage.setItem("bookhive-cart", JSON.stringify(cart));
}

// ====================
// UPDATE CART COUNT IN NAVBAR
// ====================
function updateCartCount() {
    const cartCountElement = document.getElementById("cart-count");
    if (cartCountElement) {
        const totalQuantity = cart.reduce((sum, item) => sum + item.quantity, 0);
        cartCountElement.textContent = totalQuantity;
    }
}

// ====================
// SHOW NOTIFICATION
// ====================
function showNotification(message) {
    // Create notification element
    const notification = document.createElement("div");
    notification.className = "notification";
    notification.textContent = message;
```

```
    // Add to page
    document.body.appendChild(notification);

    // Remove after 3 seconds
    setTimeout(() => {
        notification.remove();
    }, 3000);
}

// Call on page load
updateCartCount();
```

Now we need to ATTACH this function to the Add to Cart buttons. Update the renderBooks function in main.js:

```
JAVASCRIPT
// In main.js, AFTER renderBooks() runs:

// ====================
// ATTACH CLICK EVENTS TO ALL BUTTONS
// ====================
function attachAddToCartListeners() {
    const buttons = document.querySelectorAll(".add-to-cart");

    buttons.forEach(button => {
        button.addEventListener("click", function() {
            // Get the book ID from the data-id attribute
            const bookId = parseInt(this.getAttribute("data-id"));
            addToCart(bookId);
        });
    });
}

// Call after rendering books
renderBooks(books);
attachAddToCartListeners();
```

Add the notification style to style.css:

```
CSS
/* Notification toast */
.notification {
    position: fixed;
    top: 100px;
    right: 20px;
    background: var(--success);
    color: var(--white);
    padding: var(--space-sm) var(--space-md);
    border-radius: var(--border-radius);
```

```
    box-shadow: var(--shadow-md);
    z-index: 1000;
    animation: slideIn 0.3s ease-out;
}

@keyframes slideIn {
    from {
        transform: translateX(100%);
        opacity: 0;
    }
    to {
        transform: translateX(0);
        opacity: 1;
    }
}
```

Save everything. Click 'Add to Cart' on any book. A green notification slides in. The cart count in the navbar updates. Cart persists if you refresh.

Teaching Moment: THIS is real software. The student just made an interactive feature that survives page refreshes (localStorage). This is what working developers build every day.

Step 2: Search and Filter

```javascript
// js/search.js

// Get the search and filter elements
const searchInput = document.getElementById("search-input");
const categoryFilter = document.getElementById("category-filter");
const sortSelect = document.getElementById("sort");

// Function that filters and sorts books
function applyFilters() {
    let filteredBooks = [...books];  // Copy

    // 1. Apply search filter
    const searchTerm = searchInput.value.trim().toLowerCase();
    if (searchTerm) {
        filteredBooks = filteredBooks.filter(book =>
            book.title.toLowerCase().includes(searchTerm) ||
            book.author.toLowerCase().includes(searchTerm)
        );
    }

    // 2. Apply category filter
    const selectedCategory = categoryFilter.value;
    if (selectedCategory && selectedCategory !== "all") {
```

```
        filteredBooks = filteredBooks.filter(book =>
            book.category === selectedCategory
        );
    }

    // 3. Apply sorting
    const sortBy = sortSelect.value;
    switch (sortBy) {
        case "price-low":
            filteredBooks.sort((a, b) => a.price - b.price);
            break;
        case "price-high":
            filteredBooks.sort((a, b) => b.price - a.price);
            break;
        case "rating":
            filteredBooks.sort((a, b) => b.rating - a.rating);
            break;
    }

    // Re-render with filtered data
    renderBooks(filteredBooks);
    attachAddToCartListeners();  // Re-attach to new buttons

    // Show count
    showResultCount(filteredBooks.length);
}

function showResultCount(count) {
    let countDisplay = document.getElementById("result-count");
    if (!countDisplay) {
        countDisplay = document.createElement("p");
        countDisplay.id = "result-count";
        countDisplay.style.textAlign = "center";
        countDisplay.style.color = "#666";
        const grid = document.getElementById("books-container");
        grid.parentNode.insertBefore(countDisplay, grid);
    }
    countDisplay.textContent = `Showing ${count} book${count !== 1 ? 's' : ''}`;
}

// ATTACH EVENT LISTENERS
searchInput.addEventListener("input", applyFilters);
categoryFilter.addEventListener("change", applyFilters);
sortSelect.addEventListener("change", applyFilters);

// Run on page load
applyFilters();
```

Save. Type in the search box - watch books filter LIVE as you type. Change category dropdown - books update. Change sort - order changes. THIS IS A REAL APP.

Step 3: Form Validation - Signup Page

```javascript
// js/signup.js (link in signup.html)

const signupForm = document.querySelector(".signup-form");

signupForm.addEventListener("submit", function(event) {
    // STOP the page from refreshing
    event.preventDefault();

    // Get all form values
    const fullname = document.getElementById("fullname").value.trim();
    const email = document.getElementById("email").value.trim();
    const phone = document.getElementById("phone").value.trim();
    const password = document.getElementById("password").value;
    const confirmPassword = document.getElementById("confirm-password").value;
    const city = document.getElementById("city").value;
    const terms = document.getElementById("terms").checked;

    // ====================
    // VALIDATION
    // ====================

    // 1. Name check
    if (fullname.length < 3) {
        alert("Please enter your full name (at least 3 characters)");
        return;
    }

    // 2. Email check (basic regex)
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailRegex.test(email)) {
        alert("Please enter a valid email address");
        return;
    }

    // 3. Phone check (10 digits, optional field)
    if (phone && !/^[0-9]{10}$/.test(phone)) {
        alert("Phone number must be exactly 10 digits");
        return;
    }

    // 4. Password strength
```

```
    if (password.length < 8) {
        alert("Password must be at least 8 characters");
        return;
    }

    // 5. Password match
    if (password !== confirmPassword) {
        alert("Passwords don't match!");
        return;
    }

    // 6. Terms check
    if (!terms) {
        alert("Please accept the Terms and Conditions");
        return;
    }

    // ====================
    // ALL CHECKS PASSED
    // ====================

    // Create user object
    const newUser = {
        fullname: fullname,
        email: email,
        phone: phone,
        city: city,
        signupDate: new Date().toISOString()
    };

    // Save to localStorage (in real apps, this goes to a server)
    localStorage.setItem("bookhive-user", JSON.stringify(newUser));

    // Show success
    alert(`Welcome to BookHive, ${fullname}! 🎉`);

    // Redirect to home page
    window.location.href = "index.html";
});
```

Step 4: Toggle Password Visibility (Small but Pro Touch)

HTML

```
<!-- In login.html, modify password input -->
<div class="form-group">
    <label for="password">Password</label>
```

```
    <div class="password-wrapper">
        <input type="password" id="password" name="password" required>
        <button type="button" id="toggle-password" class="toggle-btn">□</button>
    </div>
</div>
```

JAVASCRIPT

```
// js/login.js
const passwordInput = document.getElementById("password");
const toggleBtn = document.getElementById("toggle-password");

toggleBtn.addEventListener("click", function() {
    if (passwordInput.type === "password") {
        passwordInput.type = "text";
        this.textContent = "□";
    } else {
        passwordInput.type = "password";
        this.textContent = "□";
    }
});
```

Step 5: Build the Cart Page Dynamically

JAVASCRIPT

```
// js/cart-page.js (link only in cart.html)

const cartItemsContainer = document.getElementById("cart-items");
const cartTotalElement = document.getElementById("cart-total");

function renderCart() {
    if (!cartItemsContainer) return;

    if (cart.length === 0) {
        cartItemsContainer.innerHTML = `
            <tr><td colspan="6" style="text-align:center; padding: 40px;">
                Your cart is empty. <a href="books.html">Browse books</a>
            </td></tr>
        `;
        cartTotalElement.textContent = "₹0";
        return;
    }

    // Generate row for each cart item
    const rowsHTML = cart.map(item => `
        <tr data-id="${item.id}">
            <td>
```

```
                <img src="${item.image}" alt="${item.title}" width="60">
            </td>
            <td>
                <strong>${item.title}</strong><br>
                <small>${item.author}</small>
            </td>
            <td>₹${item.price}</td>
            <td>
                <input
                    type="number"
                    value="${item.quantity}"
                    min="1"
                    max="10"
                    class="qty-input"
                    data-id="${item.id}"
                >
            </td>
            <td>₹${item.price * item.quantity}</td>
            <td>
                <button class="remove-btn" data-id="${item.id}">✖/button>
            </td>
        </tr>
    `).join("");

    cartItemsContainer.innerHTML = rowsHTML;

    // Calculate total
    const total = cart.reduce((sum, item) => sum + (item.price * item.quantity),
0);
    cartTotalElement.textContent = `₹${total}`;

    // Attach event listeners
    attachQuantityListeners();
    attachRemoveListeners();
}

function attachQuantityListeners() {
    const inputs = document.querySelectorAll(".qty-input");
    inputs.forEach(input => {
        input.addEventListener("change", function() {
            const id = parseInt(this.getAttribute("data-id"));
            const newQty = parseInt(this.value);

            const item = cart.find(i => i.id === id);
            if (item) {
                item.quantity = newQty;
                saveCart();
                renderCart();
```

```
                updateCartCount();
            }
        });
    });
}

function attachRemoveListeners() {
    const buttons = document.querySelectorAll(".remove-btn");
    buttons.forEach(button => {
        button.addEventListener("click", function() {
            const id = parseInt(this.getAttribute("data-id"));

            if (confirm("Remove this item from cart?")) {
                cart = cart.filter(item => item.id !== id);
                saveCart();
                renderCart();
                updateCartCount();
            }
        });
    });
}

// Render on page load
renderCart();
```

Topics Taught in This Phase

- addEventListener() - the modern way to attach events
- Click events on buttons
- input event for live filtering (fires on every keystroke)
- change event for selects and checkboxes
- submit event with event.preventDefault()
- Getting form values: .value, .checked
- Form validation with regex
- Creating elements: document.createElement()
- appendChild(), remove()
- localStorage.setItem and getItem
- JSON.stringify() and JSON.parse()
- window.location.href for navigation
- setTimeout() for delayed actions
- Dynamic rendering with map() + innerHTML
- Re-attaching event listeners after re-render

Today's Tasks for Students

- Implement Add to Cart functionality.
- Add live search and filtering to books page.
- Implement signup form validation.
- Build the dynamic cart page with quantity controls.
- Add password visibility toggle.
- Test: add items, refresh page, items should still be in cart.
- Commit: 'Phase 11 - Cart, search, filter, form validation working'.

> **BookHive is Now FUNCTIONAL!**
>
> Students can browse books, search and filter, add to cart, manage quantities, and the cart persists. This is a real, working website. Phase 12 deploys it to the internet.

PHASE 12: DEPLOYMENT & FINAL POLISH

Topic Introduction: Going Live

Your website works perfectly on your laptop. But that's it - only YOU can see it. Today we put it on the INTERNET so anyone, anywhere can access it. Free, in 10 minutes.

Step 1: Pre-Deployment Checklist

- All 8 pages render correctly.
- Navigation works between pages.
- All images load (check for broken images).
- Forms validate properly.
- Cart functionality works (add, update, remove, persist).
- Search and filter work on books page.
- Site is responsive (test on phone view in DevTools).
- No console errors (F12 → Console tab → should be clean).

Step 2: Final Polish - Loading States

```javascript
// Add to main.js - show loader when books load
function showLoader() {
    const container = document.getElementById("books-container");
    if (container) {
        container.innerHTML = '<div class="spinner"></div>';
    }
}

// Simulate loading delay (just for visual effect)
showLoader();
setTimeout(() => {
    renderBooks(books);
    attachAddToCartListeners();
}, 500);
```

Step 3: Push to GitHub

```terminal
# Check current status
git status

# Add all changes
```

```
git add .

# Commit
git commit -m "Final polish - BookHive ready for deployment"

# Push to GitHub
git push origin main
```

Step 4: Deploy with GitHub Pages (Free)

- Go to github.com and open your bookhive repository.
- Click 'Settings' (top menu).
- In the left sidebar, click 'Pages'.
- Under 'Source', select 'Deploy from a branch'.
- Branch: select 'main' and folder: '/ (root)'. Click Save.
- Wait 2-3 minutes.
- Your site is LIVE at: https://YOUR-USERNAME.github.io/bookhive

Step 5: Alternative - Deploy on Netlify

- Go to netlify.com → sign up with GitHub.
- Click 'Add new site' → 'Import an existing project'.
- Select GitHub → choose your bookhive repo.
- Click 'Deploy site'.
- In 30 seconds, you get a URL like amazing-tesla-3a8b9.netlify.app.
- Go to Site Settings → Change site name → pick something better like 'bookhive-girish'.

Step 6: Custom Domain (Optional)

Want bookhive.in instead of bookhive.netlify.app?

- Buy a domain from GoDaddy / Namecheap / Hostinger (₹100-1000/year).
- In Netlify, go to Domain settings → Add custom domain.
- Follow the DNS instructions provided.
- Within 24 hours, your site is on a custom domain.

Step 7: Final Polish Checklist

SEO Polish

- Each page has a unique <title>.
- Each page has a meta description.
- All images have alt attributes.

- Add a favicon (favicon.io can generate one).

Performance Polish

- All images use loading="lazy".
- Compress images using tinypng.com (saves 50%+ size).
- Minify CSS (optional - use cssminifier.com).
- Run Lighthouse audit in Chrome DevTools.

Professional README

Update your README.md to be impressive:

```
MARKDOWN
# 📚 BookHive - Online Bookstore

> A complete, responsive online bookstore built from scratch using
> HTML, CSS, and JavaScript.

## 🌐 Live Demo
🔗 [Visit BookHive](https://your-bookhive-url.netlify.app)

## 📸 Features

- **8 Connected Pages**: Home, Books, Single Book, Cart, Login, Signup, About,
Contact
- **Dynamic Book Catalog**: Books rendered from JavaScript data
- **Smart Search & Filter**: Live filtering by name, category, and price
- **Shopping Cart**: Add, update quantity, remove with localStorage persistence
- **Form Validation**: Signup and contact forms with proper validation
- **Fully Responsive**: Works seamlessly on mobile, tablet, and desktop
- **Modern Animations**: Smooth transitions and hover effects
- **Indian Context**: Local pricing in ₹, Indian authors and cities

## 🛠 Tech Stack

- **HTML5**: Semantic markup with proper structure
- **CSS3**:
  - CSS Variables for consistent theming
  - Flexbox for navigation and card layouts
  - CSS Grid for responsive book catalog
  - Media queries for mobile responsiveness
  - Keyframe animations
- **JavaScript (ES6+)**:
  - DOM manipulation and event handling
  - Array methods (map, filter, reduce)
  - Template literals for dynamic HTML
  - localStorage for cart persistence
  - Form validation with regex
```

```
- **Deployment**: Netlify (or GitHub Pages)

## 📁 Project Structure

\`\`\`
bookhive/
├── index.html
├── books.html
├── cart.html
├── login.html, signup.html
├── about.html, contact.html
├── css/
│   ├── style.css
│   ├── books.css
│   └── responsive.css
├── js/
│   ├── main.js
│   ├── books.js  (data)
│   ├── cart.js
│   └── search.js
└── images/
\`\`\`

## 🎓 What I Learned

Built this project to practice and demonstrate:
- Building production-grade websites without frameworks
- Working with browser APIs (localStorage, DOM)
- Creating responsive layouts that work on all devices
- Writing clean, maintainable code with proper separation of concerns

## 👤 Author

**[Your Name]** - Aspiring Full Stack Developer
- 🌐 Portfolio: [your-portfolio.com]
- 💼 LinkedIn: [linkedin.com/in/yourname]
- 🐦 Twitter: [@yourhandle]

## 📸 Screenshots

(Add screenshots of home page, books page, cart page here)

## 📄 License

Free to use for learning purposes.
```

Step 8: Add to Your Resume

Add this project to your resume IMMEDIATELY:

```
RESUME
PROJECTS

BookHive - Online Bookstore | HTML, CSS, JavaScript
 bookhive-yourname.netlify.app | github.com/yourname/bookhive

• Built an 8-page responsive online bookstore from scratch
• Implemented dynamic book catalog with live search and filtering using
  ES6 array methods (map, filter, reduce)
• Created persistent shopping cart with localStorage and dynamic
  quantity management
• Designed mobile-first responsive layouts using CSS Grid and Flexbox
• Deployed to Netlify with optimized performance (Lighthouse 95+)
```

Career Reality Check

This project demonstrates that you can build a real, working website end-to-end. This is more impressive to recruiters than 10 tutorials. The live link is your proof. Put it everywhere.

Step 9: Future Improvements (Talk Points for Interviews)

If asked 'what would you add next?', mention:

- Backend with Node.js and Express for real user accounts.
- MongoDB to store users and orders persistently.
- Payment integration with Razorpay or Stripe.
- Convert to React for a more modern architecture.
- Add admin dashboard for managing books.
- Email notifications for orders.
- Real product images via Cloudinary.
- Search with autocomplete.

Today's Tasks for Students

- Complete the pre-deployment checklist.
- Update README.md to be impressive.
- Deploy to GitHub Pages or Netlify.
- Run Lighthouse audit (aim for 90+ on all metrics).
- Share the live URL on WhatsApp/LinkedIn/Twitter.
- Add to your resume immediately.
- Final commit: 'BookHive v1.0 - Live deployment complete'.

Course Complete - What Students Have Built

Over these 12 phases, students went from zero to a deployed, working website. Here's the complete topic coverage:

HTML Topics Mastered

- HTML5 boilerplate and meta tags
- All semantic tags (header, nav, main, section, article, aside, footer)
- Heading hierarchy and text formatting
- Links, images with alt attributes, picture elements
- Forms with all input types
- Lists - unordered, ordered, definition
- Tables with thead, tbody, tfoot, colspan, rowspan
- data-* attributes for JS integration
- Accessibility basics (alt, labels, ARIA)
- SEO meta tags and Open Graph

CSS Topics Mastered

- CSS reset and box-sizing: border-box
- CSS variables for theming
- All selector types (element, class, ID, pseudo)
- Box model (margin, padding, border)
- Typography and font styling
- Colors, gradients, transparency
- Flexbox for one-dimensional layouts
- CSS Grid for two-dimensional layouts
- Responsive design with media queries
- Transitions and transforms
- @keyframes animations
- Position: sticky, fixed, absolute, relative
- Mobile-first design principles

JavaScript Topics Mastered

- Variables (const, let), data types
- Template literals for dynamic strings
- Functions and arrow functions
- Objects and arrays
- Array methods (map, filter, reduce, find, sort)
- DOM selection (querySelector, getElementById)
- DOM manipulation (textContent, innerHTML, createElement)

- Event handling (click, input, change, submit)
- Form validation with regex
- localStorage for data persistence
- JSON.stringify and JSON.parse
- Dynamic HTML generation
- Conditional logic and loops
- Event delegation patterns

Industry Skills Demonstrated

- Project planning before coding
- Wireframing and architecture design
- Folder structure organization
- Git workflow with meaningful commits
- Reading documentation
- Debugging with browser DevTools
- Performance optimization (lazy loading, image compression)
- SEO and accessibility
- Deployment to production
- Writing professional README documentation

Where to Go Next

BookHive is built with HTML, CSS, and JavaScript only. Next steps:

Option 1: Make it Modern with React (Recommended)

Rebuild BookHive using React. Same UI, but with components, props, state, and React Router. This is what Week 4-5 of the MERN curriculum covers.

Option 2: Add a Backend with Node.js

Add user accounts that actually work, save orders to a database, add admin dashboard. This is Week 6-7 of MERN.

Option 3: Build More Projects

- Recipe website (with categories, search)
- Personal blog (with admin to add posts)
- Movie database (using TMDB API)
- Weather dashboard (using OpenWeather API)

Final Words for Trainers

This project-based approach works because students see EVERY topic in CONTEXT. They don't ask 'why am I learning this?' - they see WHY because they USE it immediately. Each phase builds on the previous one. By the end, they have a real, deployable website.

Encourage students to PERSONALIZE BookHive - different theme, different niche (cooking website, fitness tracker, travel blog). The skills transfer; the project doesn't have to be a bookstore.

Most importantly: every student walks away with a LIVE URL they can show in interviews. That's worth more than 100 tutorials they 'completed'.

Final Note

BookHive is just the START. The real value of this project is teaching students HOW TO BUILD. Once they internalize the build process - plan, structure, style, interact, deploy - they can build anything with any technology.